한국폴리텍대학 1대학 서울정수캠퍼스

파이썬
(하이테크과정)

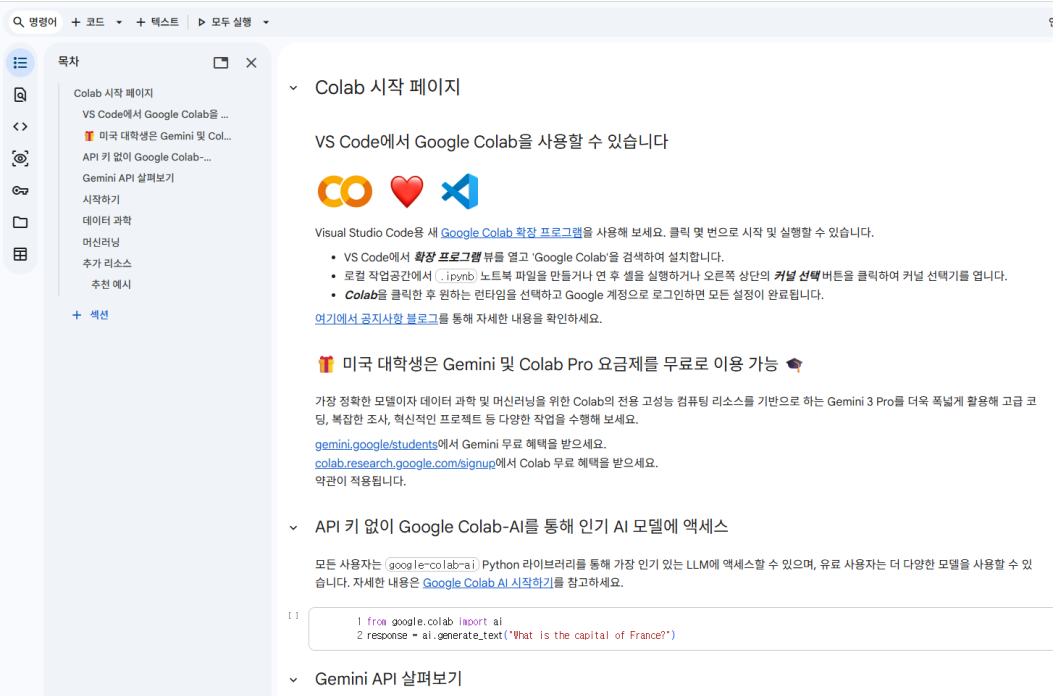
담당교수 : 김찬수

구글 코랩(colab)

실습 환경

구글 코랩(https://colab.research.google.com/)

- 구글에서 무료로 제공 (구글 계정만 있으면, 누구나 사용가능)
- 웹브라우저에서 파이썬(Python) 작성 및 실행
  - 크롬(Chrome) 브라우저 추천



무단 수정 및 배포 금지

실습 환경

구글 코랩

- GPU 기반 딥러닝 환경 무료 제공
  - 주피터 노트북(Python3, R, Julia)
  - T4 GPU, v5e-1 TPU
- 환경 설정 최소화
  - 관련 라이브러리가 자동으로 설치되어 제공
- 클라우드 기반
  - 코드 공유 및 수정 수월
  - 인터넷 브라우저만으로도 동작
- 사용 제약
  - 최대 세션 유지시간 12시간
  - 상업적 용도 불가

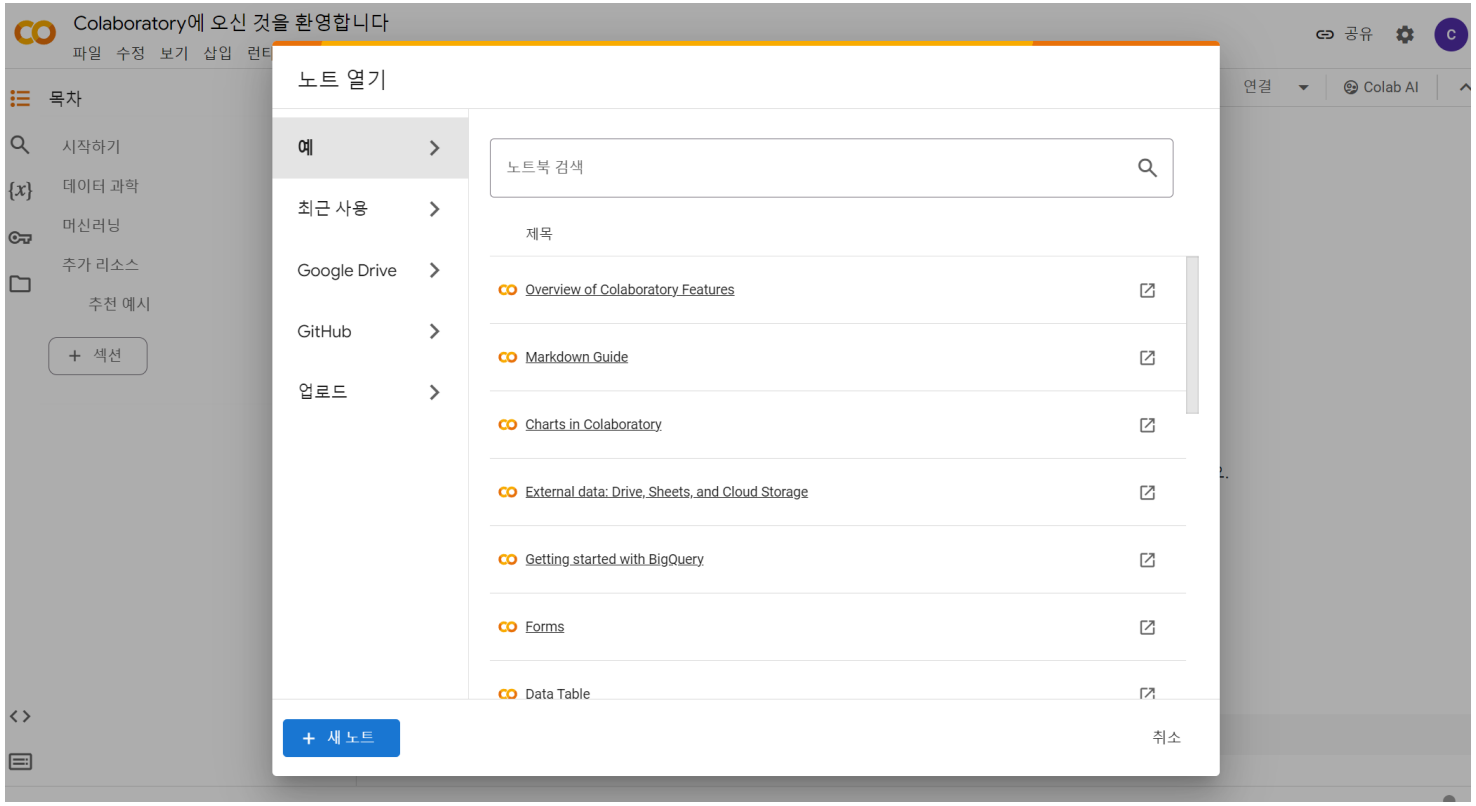
구분	무료	Pro 이상	비고
CPU	Intel® Xeon® CPU @2.2/2.3GHz	Intel® Xeon® CPU @2.2/2.3GHz	-
GPU	T4 등	H100, G4, A100, L4 등	-
		TPU 우선 할당	사용시간, 전체 이용량에 따라 가변적
RAM	12GB	25GB/ 12GB	용량 변경 설정 필요
세션 유지시간	12시간	24시간	유지시간 보장하지 않음

무단 수정 및 배포 금지

구글 코랩 시작하기

새노트 만들기 – 방법 1

- 'https://colab.research.google.com/' 접속
- 'Google Drive' → '새노트' 선택

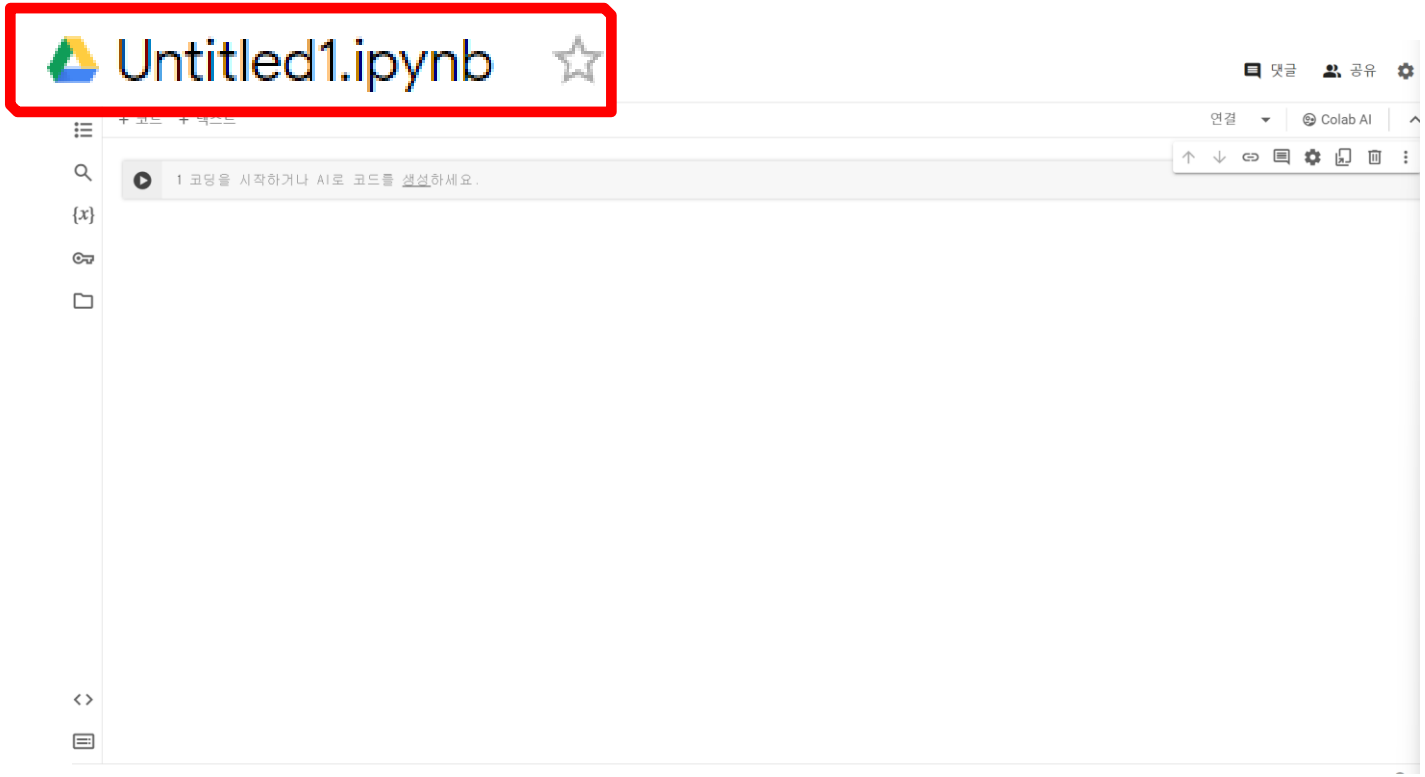


무단 수정 및 배포 금지

구글 코랩 시작하기

새노트 만들기 – 방법 1

- 노트 명칭 변경
- 모든 내용은 구글 드라이버에 저장됨

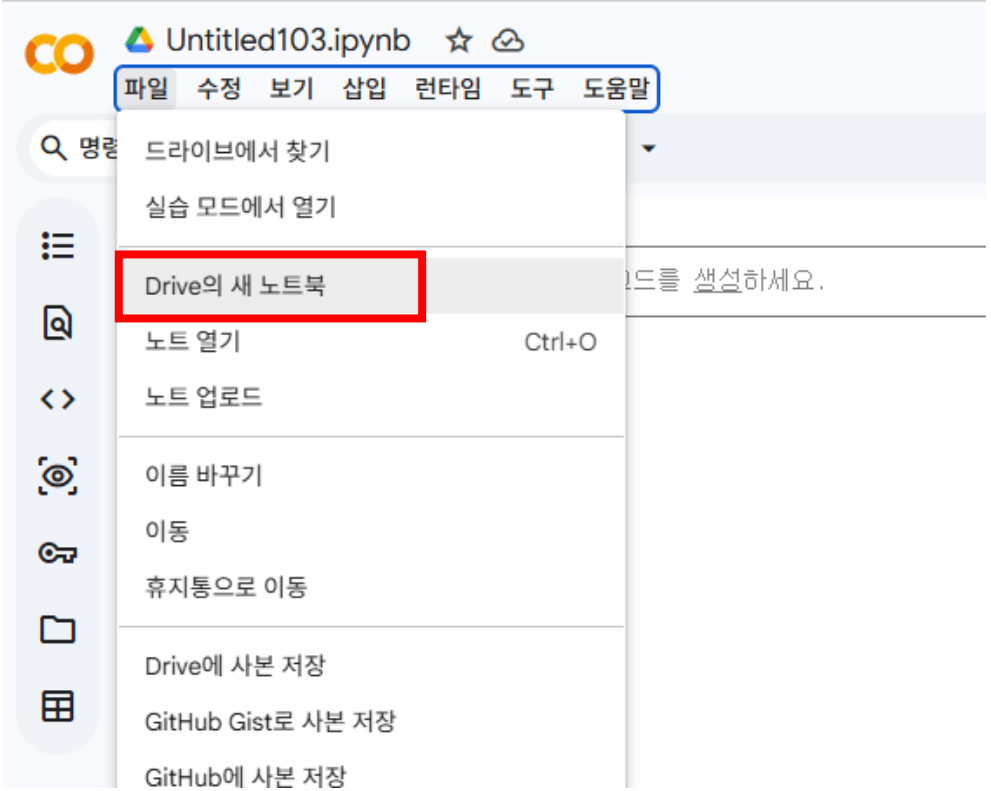


무단 수정 및 배포 금지

구글 코랩 시작하기

새노트 만들기 – 방법 2

- 'https://colab.research.google.com/' 접속
- '파일' → 'Drive의 새 노트북' 선택

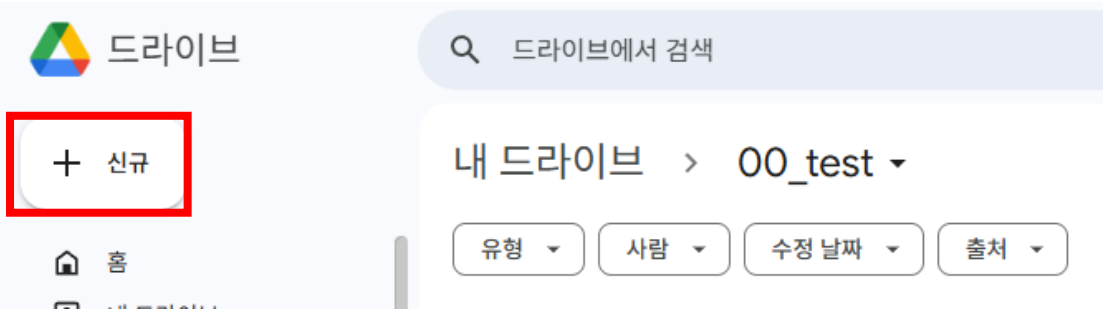


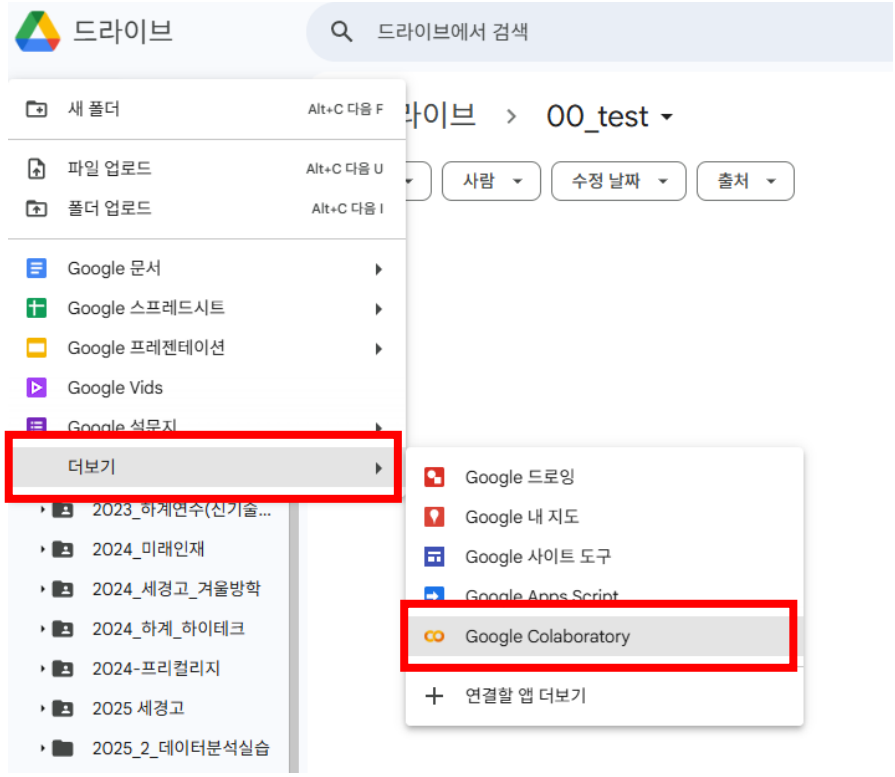
무단 수정 및 배포 금지

구글 코랩 시작하기

새노트 만들기 – 방법 3

- 구글 드라이브 → 내드라이브 → (마우스 우클릭)새폴더 선택
- 생성된 폴더 → 좌측 상단 "+신규 " → 더보기 → "Google colaboratory" 선택



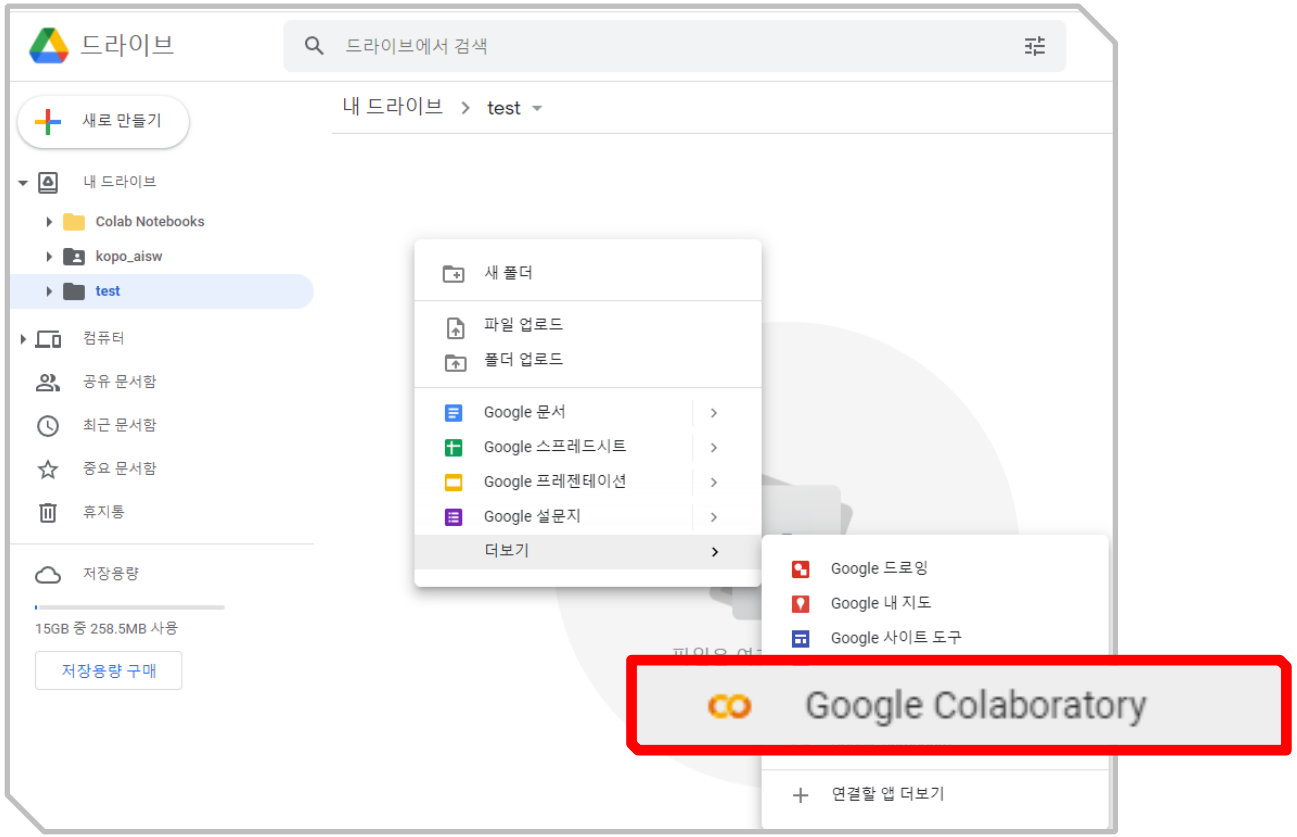


무단 수정 및 배포 금지

구글 코랩 시작하기

새노트 만들기 – 방법 3

- 생성된 폴더 선택 → (빈영역 마우스우클릭)더보기 → Google Colaboratory 선택

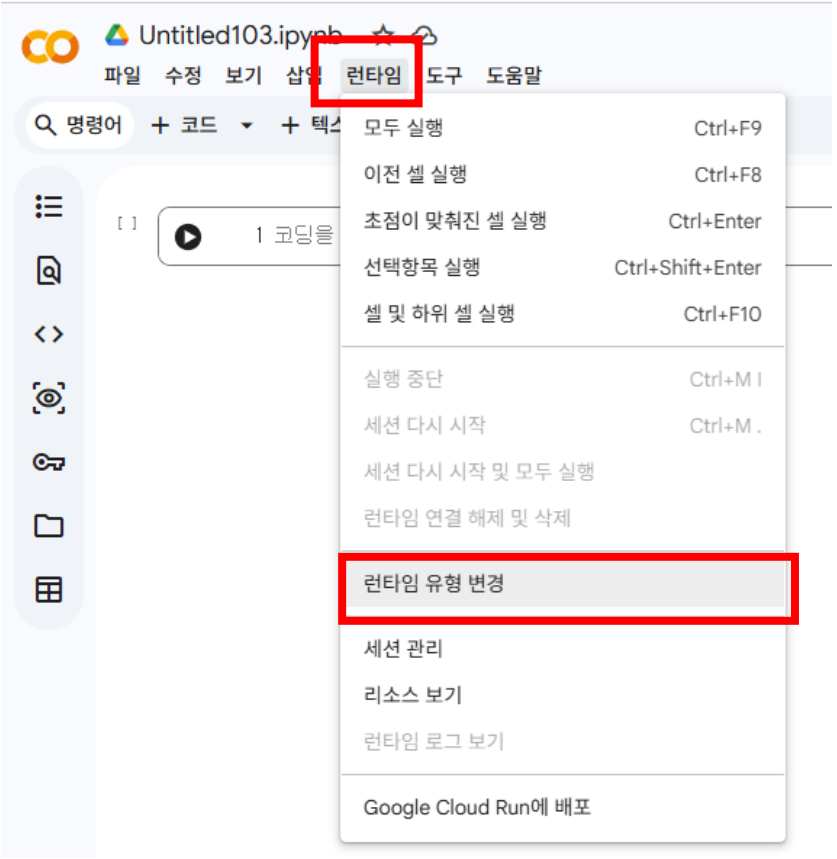


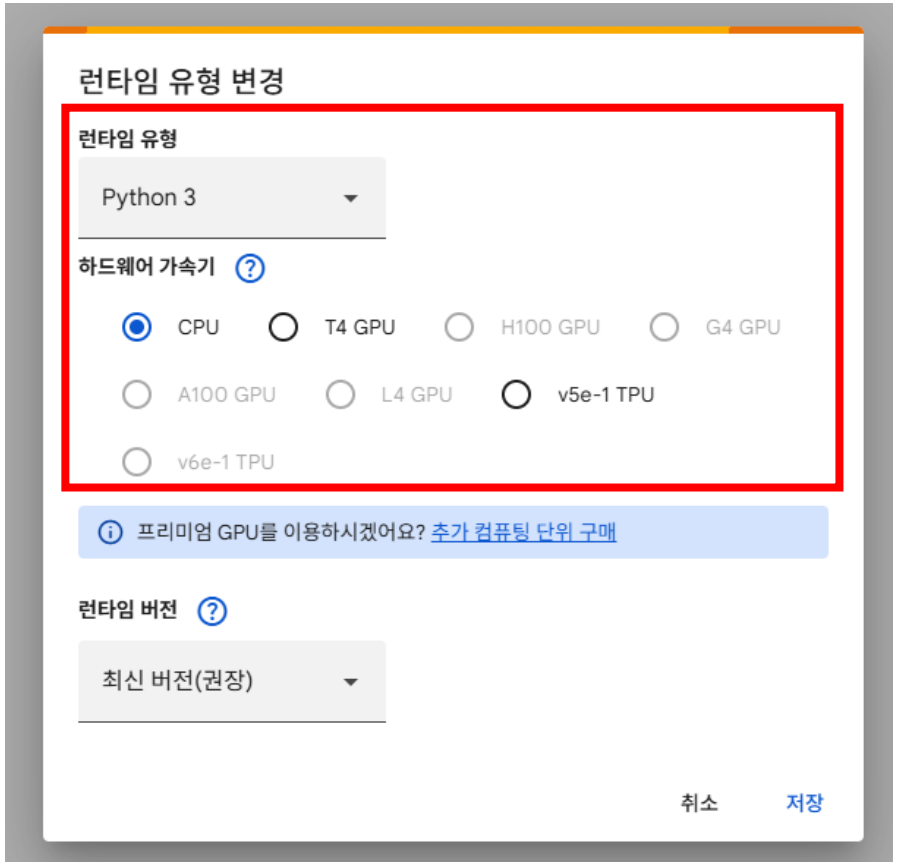
무단 수정 및 배포 금지

구글 코랩 환경 설정

런타임 유형 변경

- CPU, GPU, TPU





무단 수정 및 배포 금지

구글 코랩 환경 설정

환경 확인 명령어

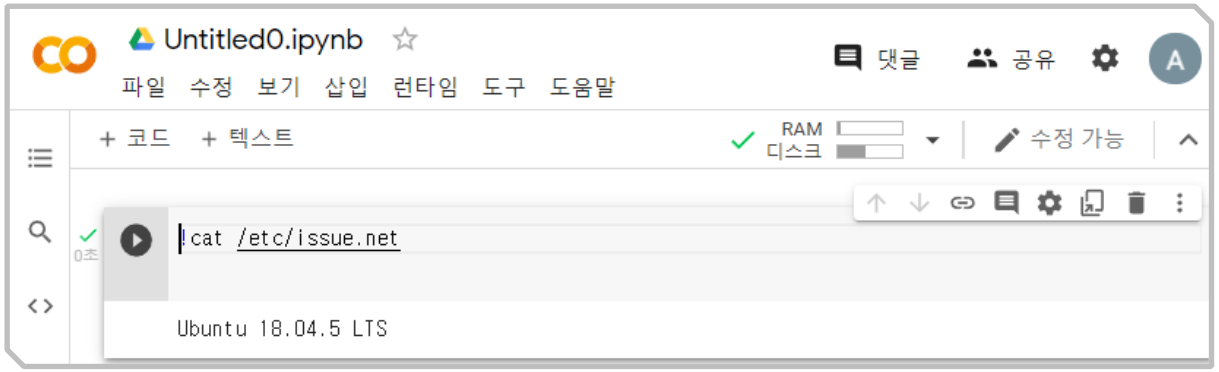
용도	설명
OS 확인	!cat /etc/issue.net
CPU 확인	!cat /proc/cpuinfo
메모리 확인	!cat /proc/meminfo
하드디스크 확인	!df -h
GPU 확인	!nvidia-smi
파이썬버전 확인	!python --version

무단 수정 및 배포 금지

구글 코랩 명령어

명령어

- 기본 수행 단위 : 셀
- 코딩 방법
  - 셀에 커서를 위치시키고 명령어 작성
  - 키보드 'Enter' 입력하면 줄 바꿈
- 셀 수행 방법
  - 셀 좌측 버튼 클릭 (ctrl + enter)
    - 해당 셀만 수행



무단 수정 및 배포 금지

구글 코랩 명령어

셀 수행 방법

- alt + enter
  - 해당 셀 수행 및 다음 셀 생성(커서 이동 포함)
- shift + enter
  - 해당 셀 수행 및 커서를 다음 셀로 이동
  - 다음 셀이 없다면 셀 생성함
- ctrl + F10: 노트의 모든 셀 실행
- ctrl + M + I: 실행 중단

무단 수정 및 배포 금지

구글 코랩 명령어

셀 수행 방법

- 셀 추가
  - 셀 추가하고자 하는 곳에 마우스 위치 → 코드 선택
  - ctrl + M + A(위쪽에 셀 추가), ctrl + M + B (아래쪽에 셀 추가)
  - esc + A, esc + B
- 셀 삭제
  - 삭제하고자 하는 셀 클릭 → 휴지통 선택
  - ctrl + M + D
- 셀 이동
  - 이동하고자 하는 셀 클릭 → 화살표 선택
  - ctrl + M + J(아래로 이동), ctrl + M + K (위로 이동)

무단 수정 및 배포 금지

파이썬 소개

파이썬 주요 특징

인터프리터 언어(스크립트 언어)

- 원시코드를 기계어로 변환하는 과정 없이 줄(라인) 단위로 바로 명령어 실행
- 컴파일 언어 : 원시코드 → 기계어 변환 → 실행파일 생성

동적 타입 언어

- 변수의 자료 유형(data type)을 실행 중 정하거나, 정해진 자료 유형 변경 가능
- 프로그래밍 유연성 제공

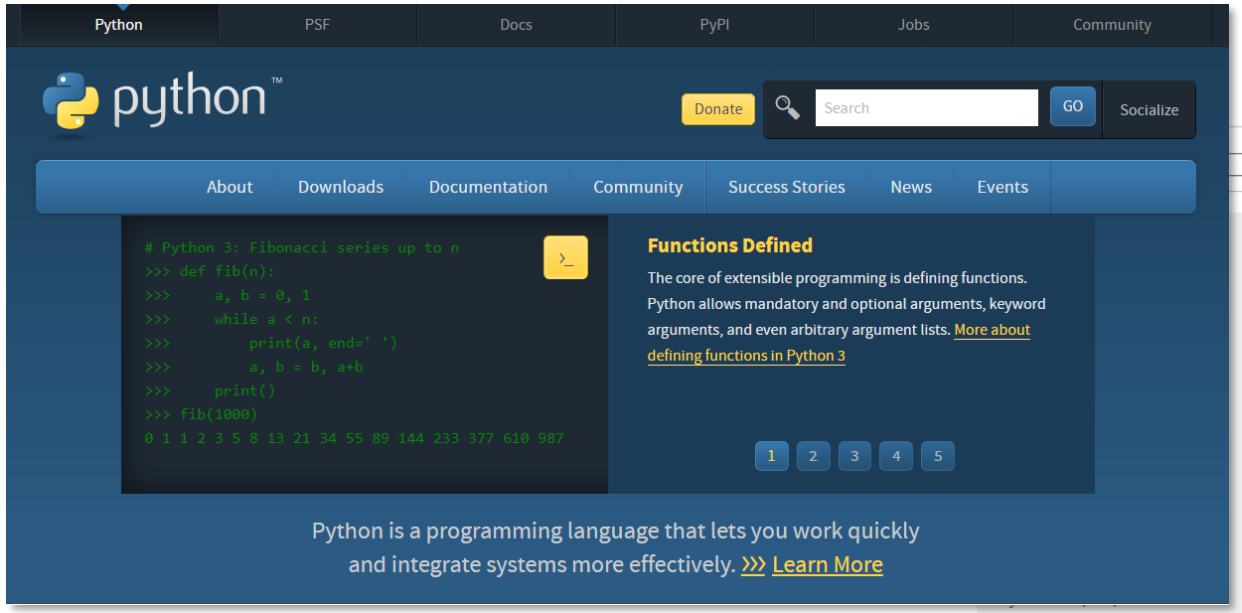
플랫폼 독립적 언어

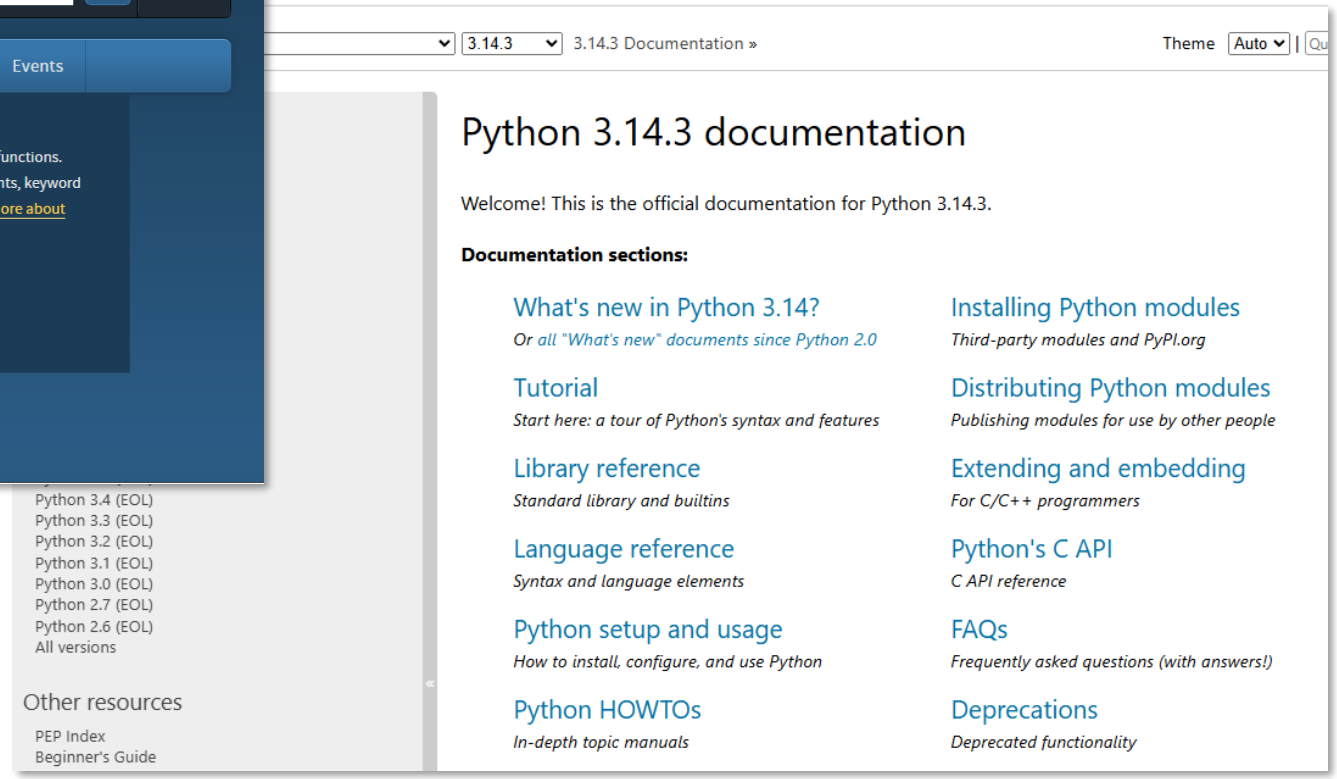
- 대부분의 운영체계에서 모두 동작
- 윈도우, 리눅스, 맥 등

무단 수정 및 배포 금지

주요 참고 사이트

- https://www.python.org/
- https://docs.python.org/





무단 수정 및 복제, 배포를 금지.

Python keywords

- 파이썬에서 예약된 단어(words)
  - 파이썬 언어의 구문(syntax)과 구조(structure)를 정의하는데 사용
- 변수명, 함수명 또는 식별자(identifiers)로 사용할 수 없음

Keywords in Python programming language				
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

무단 수정 및 복제, 배포를 금지.

Rules for writing identifiers(식별자, 변수이름)

- 소문자, 대문자, 숫자, _ (underscore)로 조합 가능 (**대소문자 구분함**)
  - 일반적으로 소문자로 변수이름 작성, 대문자는 특별한 경우(클래스 이름 등)에 사용
  - _ 로 시작하는 이름은 클래스 내부용으로 주로 사용, _ _는 특별한 의미를 가지기에 사용 지양
  - 예시: myClass, var_1, print_value
- 숫자로 시작해서는 안됨
  - 위반 : 1var
  - 가능 : var1
- 키워드(파이썬 문법에서 사용하는 식별자)를 식별자로 사용할 수 없음
  - Global = 10, if = 5
- 특정 심벌을 사용할 수 없음
  - !, @, #, $, % 등
- 식별자의 길이는 상관없음

c=10 vs count=10

thisisalongvariable vs **this_is_a_long_variable** vs **ThisIsALongVariable**

무단 수정 및 복제, 배포를 금지.

Python statement

- 명령문
  - 예) a=1 : assignment statement
- Multi-line statement
  - 역 슬래시(₩)
  - 세미콜론(;) 기호로 다중 명령문 가능
  - 자료형으로 처리: 괄호 ( ), 대괄호 [ ], 중괄호 { }

```
a = (1 + 2 + 3 +
     4 + 5 + 6 +
     7 + 8 + 9)
```

```
a = 1 + 2 + 3 + \
    4 + 5 + 6 + \
    7 + 8 + 9
```

```
a = 1; b = 2; c = 3
```

무단 수정 및 복제, 배포를 금지.

Python indentation, comments

- 들여쓰기(indentation)
  - c, c++, java 등은 중괄호(braces) { } 로 코드의 블록을 정의하지만
  - 파이썬에서는 들여쓰기로 코드의 블록을 정의함
  - 일반적으로 4 칸을 띄움(tab 키 사용)

```
for i in range(1,11):
    print(i)
    if i == 5:
        break
```

```
if True:
    print('Hello')
    a = 5
```

vs

```
if True: print('Hello'); a = 5
```

- 주석(comments)
  - 해쉬(hash) 기호 #으로 구성
  - ' ' ' 또는 " " " 기호로 다중 주석 처리

```
#This is a comment
#print out Hello
print('Hello')
```

```
"""This is also a
perfect example of
multi-line comments"""
```

무단 수정 및 복제, 배포를 금지.

Docstrings(documentation strings) in python

- 함수(function), 방법(method), 클래스(class), 모듈(module)의 정의 이후에 사용하는 문자열 리터럴(literals)
- __doc__ 속성(attribute)으로 객체와 연결됨

```
def double(num):
    """Function to double the value"""
    return 2*num
print(double.__doc__)
```

무단 수정 및 복제, 배포를 금지.

Python variables(변수)

- 메모리에 데이터를 저장하기 위해 사용되는 지정된 위치

number = 10 : 변수를 number 라는 이름으로 선언
number = 1.1 : 변수의 내용물은 언제든 변경 할 수 있음

- 변수에 데이터/값을 할당하기 위해서는 = 기호 사용
  - 파이썬은 type-inferred 언어로, 변수 유형을 명시적으로 정의할 필요 없음

```
website = "kopo.ac.kr"
print(website)
```

```
website = "kopo.ac.kr"
print(website)
website = "naver.com"
print(website)
Website = 1
print(website)
```

```
a, b, c = 10, 5.2,"kopo"
print(a)
print(b)
print(c)
```

```
a = b = c = "kopo"
print(a)
print(b)
print(c)
```

무단 수정 및 복제, 배포를 금지.

Python constants(상수)

- 모듈에서 선언되고 할당
  - 모듈 : import를 통해 가져온 변수, 함수 등이 포함된 새 파일
- 상수는 대문자, _ 기호의 조합으로 작성

```
# constant.py

PI = 3.14
```

```
#main.py

import constant

printf(constant.PI)
```

- 상수 사용하지 않음. 재할당을 방지하는 기능이 없음

무단 수정 및 복제, 배포를 금지.

Literal

- 변수 또는 상수에 대입하는 원시 데이터(raw data)
  - Numeric : int, float, complex
  - String
  - Boolean: True, False

```
a = 0b1010 #Binary Literals
b = 100 #Decimal Literal
c = 0o310 #Octal Literal
d = 0x12c #Hexadecimal Literal

#Float Literal
float_1 = 10.5
float_2 = 1.5e2

#Complex Literal
x = 3.14j

print(a, b, c, d)
print(float_1, float_2)
print(x, x.imag, x.real)
```

```
x = (1 == True)
y = (1 == False)
a = True + 4
b = False + 10

print("x is", x)
print("y is", y)
print("a:", a)
print("b:", b)
```

```
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline string
with more than one line code."""
unicode = u"\u00dcnic\u00f6de"
raw_str = r"raw \n string"

print(strings)
print(char)
print(multiline_str)
print(unicode)     # Ünicöde
print(raw_str)
```

무단 수정 및 복제, 배포를 금지.

Special literal & literal collections

- None : 생성되지 않았음을 지정
- 리터럴 모음 방법
  - list
  - tuple
  - dict
  - set

```
drink = "Available"
food = None

def menu(x):
    if x == drink:
        print(drink)
    else:
        print(food)

menu(drink)
menu(food)
```

```
fruits = ["apple", "mango", "orange"] #list
numbers = (1, 2, 3) #tuple
alphabets = {'a':'apple', 'b':'ball', 'c':'cat'} #dictionary
vowels = {'a', 'e', 'i' , 'o', 'u'} #set

print(fruits)
print(numbers)
print(alphabets)
print(vowels)
```

무단 수정 및 복제, 배포를 금지.

Python data types

- 파이썬 프로그래밍의 모든 것은 객체이고, 이들은 데이터 유형을 가짐
  - 데이터 형(data types)은 실제로는 클래스(class)이고, 변수(variable)는 클래스의 인스턴스(instance)(객체(object)) 임
- 종류
  - number
  - string
  - bool
  - list
  - tuple
  - set
  - dictionary

무단 수정 및 복제, 배포를 금지.

Python data types : number

- 정수 : 메모리 가용한 범위내에서 길이 제한 없음
- 실수 : 소수점 이하 15자리까지 정확

```
# Ouput : 5 is of type <class 'int'>
a = 5
print(a, "is of type", type(a))

# Ouput : 2.0 is of type <class 'float'>
a = 2.0
print(a, "is of type", type(a))

# Ouput : (1+2j) is complex number? True
a = 1+2j
print(a, "is complex number?", isinstance(1+2j,complex))
```

무단 수정 및 복제, 배포를 금지.

Python data types : list

- Item의 순서가 지정된 목록(list)
- list의 item이 다른 유형도 가능
- 선언 : 대괄호(brackets) [ ] 내부에 쉼표(,)로 구분하여 item 나열
- item의 인덱스(index)는 0 부터 시작
- item 변경 가능

```
a = [5, 10, 15, 20, 25, 30, 35, 40]

# Output : a[2] = 15
print("a[2] = ", a[2])

# Output : a[0:3] = [5, 10, 15]
print("a[0:3] = ", a[0:3])

# Output : a[5:] = [30, 35, 40]
print("a[5:] = ", a[5:])

# Output : [100, 10, 15, 20, 25, 30, 35, 40]
a[0] = 100; print(a)
```

무단 수정 및 복제, 배포를 금지.

Python data types : tuple

- list와 기능 동일하지만, 선언 이후에는 변경 불가능
  - 일반적으로, list 보다 속도가 빠름
- 선언 : 괄호(parentheses) ( ) 내부에 쉼표(,)로 구분하여 item 나열
- item의 인덱스(index)는 0 부터 시작

```
t = (5,'program', 1+3j)

# t[1] = 'program'
print("t[1] = ", t[1])

# t[0:3] = (5, 'program', (1+3j))
print("t[0:3] = ", t[0:3])

# Generates error
# Tuples are immutable
t[0] = 10
```

무단 수정 및 복제, 배포를 금지.

Python data types : string

- 유니코드(Unicode) 문자들의 나열(sequence)
- list, tuple 처럼 슬라이싱(slicing) 연산자 [ ] 를 사용하여 접근가능
- slicing operation을 통해 변경 불가능
- enumerate()을 통해 인덱스와 값(글자)의 쌍 생성 가능

```
s = "This is a string"
print(s)
s = '''A multiline
string'''
print(s)
```

```
s = 'Hello world!'

# s[4] = 'o'
print("s[4] = ", s[4])
# s[6:11] = 'world'
print("s[6:11] = ", s[6:11])
# Generates error : Strings are
immutable in Python
s[5] ='d'
```

```
str = 'cold'

# enumerate()
list_enumerate = list(enumerate(str))
print('list(enumerate(str) = ', list_enumerate)

#character count
print('len(str) = ', len(str))
```

무단 수정 및 복제, 배포를 금지.

Python data types : string

- Escape sequences

Escape Sequence	Description
\ newline	Backslash and newline ignored
\\	Backslash
\'	Single quote
\"	Double quote
\a	ASCII Bell
\b	ASCII Backspace
\f	ASCII Formfeed
\n	ASCII Linefeed
\r	ASCII Carriage Return
\t	ASCII Horizontal Tab
\v	ASCII Vertical Tab
\ooo	Character with octal value ooo
\xHH	Character with hexadecimal value HH

무단 수정 및 복제, 배포를 금지.

Python data types : set

- 고유한(unique) item 들의 정렬되지 않은 모음
- 선언 : 중괄호(braces) { } 내부에 쉼표(,)로 구분하여 item 나열
- slicing operation을 통해 접근 불가
- 변경 가능한 아이템(list 등)은 가질 수 없음
- 아이템 추가, 제거 가능
  - add(), update()
  - discard(), remove()

```
a = {1,2,2,3,3,3}
print(a)
```

```
a = {5,2,3,1,4}

# printing set variable
print("a = ", a)
# data type of variable a
print(type(a))
```

```
# initialize my_set
my_set = {1, 3}
print(my_set)

# add an element
# Output: {1, 2, 3}
my_set.add(2)
print(my_set)

# add multiple elements
# Output: {1, 2, 3, 4}
my_set.update([2, 3, 4])
print(my_set)

# add list and set
# Output: {1, 2, 3, 4, 5, 6, 8}
my_set.update([4, 5], {1, 6, 8})
print(my_set)
```

무단 수정 및 복제, 배포를 금지.

Python data types : set

- 다양한 내장 함수 지원

Function	Description
all()	Returns True if all elements of the set are true (or if the set is empty).
any()	Returns True if any element of the set is true. If the set is empty, returns False.
enumerate()	Returns an enumerate object. It contains the index and value for all the items of the set as a pair.
len()	Returns the length (the number of items) in the set.
max()	Returns the largest item in the set.
min()	Returns the smallest item in the set.
sorted()	Returns a new sorted list from elements in the set(does not sort the set itself).
sum()	Returns the sum of all elements in the set.

무단 수정 및 복제, 배포를 금지.

Python data types : set

Method	Description
add()	Adds an element to the set
clear()	Removes all elements from the set
copy()	Returns a copy of the set
difference()	Returns the difference of two or more sets as a new set
difference_update()	Removes all elements of another set from this set
discard()	Removes an element from the set if it is a member. (Do nothing if the element is not in set)
intersection()	Returns the intersection of two sets as a new set
intersection_update()	Updates the set with the intersection of itself and another
isdisjoint()	Returns True if two sets have a null intersection
issubset()	Returns True if another set contains this set
issuperset()	Returns True if this set contains another set
pop()	Removes and returns an arbitrary set element. Raises KeyError if the set is empty
remove()	Removes an element from the set. If the element is not a member, raises a KeyError
symmetric_difference()	Returns the symmetric difference of two sets as a new set
symmetric_difference_update()	Updates a set with the symmetric difference of itself and another
union()	Returns the union of sets in a new set
update()	Updates the set with the union of itself and others

무단 수정 및 복제, 배포를 금지.

Python data types : dictionary

- Key-value 쌍(pair)의 정렬되지 않은 모음
- 데이터의 양이 많을 때 주로 사용함
  - 데이터 검색(retrieving data)에 최적화 되어 있음
  - Key를 통해 value 검색
- 선언 : 중괄호(braces) { } 내부에 쉼표(,)로 구분하여 key:value 나열
  - key, value은 데이터 형에 무관

```
d = {1:'value','key':2}
print(type(d))

print("d[1] = ", d[1])

print("d['key'] = ", d['key'])

# Generates error
print("d[2] = ", d[2])
```

무단 수정 및 복제, 배포를 금지.

Python type conversion and type casting

- Implicit (암시/암묵적 변경)
  - 파이썬(python interpreter)이 자동으로 형변환 실시
  - 사용자가 개입이 필요치 않음

```
num_int = 123
num_flo = 1.23

num_new = num_int + num_flo

print("datatype of num_int:", type(num_int))
print("datatype of num_flo:", type(num_flo))

print("Value of num_new:", num_new)
print("datatype of num_new:", type(num_new))
```

```
num_int = 123; num_str = "456"

print("Data type of num_int:",type(num_int))
print("Data type of num_str:",type(num_str))

# Error
print(num_int+num_str)
```

무단 수정 및 복제, 배포를 금지.

Python type conversion and type casting

- Explicit (명시적 변경) / typecasting
  - 사용자가 원하는 형태로 변경
  - int(), float(), str() 등

```
num_int = 123
num_str = "456"

print("Data type of num_int:", type(num_int))
print("Data type of num_str before Type Casting:", type(num_str))

num_str = int(num_str)
print("Data type of num_str after Type Casting:", type(num_str))

num_sum = num_int + num_str

print("Sum of num_int and num_str:", num_sum)
print("Data type of the sum:", type(num_sum))
```

무단 수정 및 복제, 배포를 금지.

Python input, output and import

- print()
  - 화면으로 데이터를 출력하기 위한 함수(function)
  - 원형 : `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`

```
print('This sentence is output to the screen’)

a = 5
print('The value of a is', a)
# output : The value of a is 5
```

```
print(1, 2, 3, 4)
print(1, 2, 3, 4, sep='*')
print(1, 2, 3, 4, sep='#', end='&’)
# output : 1 2 3 4
# output : 1*2*3*4
# output : 1#2#3#4&
```

무단 수정 및 복제, 배포를 금지.

Python input, output and import

- str.format()
  - 화면 출력에 대한 표현 방식의 편의 제공
  - 문자열과 입력 객체 정보의 조합으로 구성
  - Braces { }로 입력 객체를 표현, tuple index, keyword argement 지원
  - lower(), upper(), join(), split(), find(), replace() 등

```
x = 5; y = 10
print('The value of x is {} and y is {}'.format(x,y))
# output : The value of x is 5 and y is 10
```

```
print('I love {0} and {1}'.format('bread','butter'))
print('I love {1} and {0}'.format('bread','butter'))
```

```
print('Hello {name}, {greeting}'.format(greeting = 'Goodmorning', name = 'John'))
```

- % 지원 (c언어의 입출력 옵션)

```
x = 12.3456789
print('The value of x is %3.2f' %x)
print('The value of x is %d' %x)
```

무단 수정 및 복제, 배포를 금지.

Python input, output and import

- input()
  - 사용자로부터 데이터를 입력 받을 수 있도록 지원

```
num = input('Enter a number: ')
print(num)
print(type(num))
```

```
num = int(input('Enter a number: '))
print(num)
print(type(num))
```

무단 수정 및 복제, 배포를 금지.

Python input, output and import

- import
  - 모듈(module, filename.py)를 사용할 수 있도록 해 주는 것
  - 모듈을 가져오는 동안에, 파이썬은 sys.path 에 정의된 여러 위치를 참조함

```
import math
print(math.pi)
```

```
from math import pi
pi
```

```
import sys
sys.path
# output 예시
['', '/content',
 '/env/python',
 '/usr/lib/python37.zip', …]
```

무단 수정 및 복제, 배포를 금지.

Python operators

- 산술(arithmetic) 연산자

Operator	Meaning	Example
+	Add two operands or unary plus	x + y+ 2
-	Subtract right operand from the left or unary minus	x - y- 2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y (x to the power y)

```
x = 15; y = 4

# Output: x + y = 19
print('x + y =',x+y)

# Output: x - y = 11
print('x - y =',x-y)

# Output: x * y = 60
print('x * y =',x*y)

# Output: x / y = 3.75
print('x / y =',x/y)

# Output: x // y = 3
print('x // y =',x//y)

# Output: x ** y = 50625
print('x ** y =',x**y)
```

무단 수정 및 복제, 배포를 금지.

Python operators

- 비교(comparison) 연산자

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

```
x = 10; y = 12

# Output: x > y is
print('x > y is',x>y)

# Output: x < y is
print('x < y is',x<y)

# Output: x == y is
print('x == y is',x==y)

# Output: x != y is
print('x != y is',x!=y)

# Output: x >= y is
print('x >= y is',x>=y)

# Output: x <= y is
print('x <= y is',x<=y)
```

무단 수정 및 복제, 배포를 금지.

Python operators

- 논리(logical) 연산자

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

```
x = True
y = False

print('x and y is', x and y)

print('x or y is', x or y)

print('not x is', not x)
```

무단 수정 및 복제, 배포를 금지.

Python operators

- 비트(bitwise) 연산자

Operator	Meaning	Example (10: 00001010, 4: 00000100)
&	Bitwise AND	x & y = 0 (0000 0000)
|	Bitwise OR	x | y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
>>	Bitwise right shift	x >> 2 = 2 (0000 0010)
<<	Bitwise left shift	x << 2 = 40 (0010 1000)

무단 수정 및 복제, 배포를 금지.

Python operators

- 대입(assignment) 연산자

Operator	Example	Equivalent to	Operator	Example	Equivalent to
=	x = 5	x = 5	&=	x &= 5	x = x & 5
+=	x += 5	x = x + 5	|=	x |= 5	x = x | 5
-=	x -= 5	x = x - 5	^=	x ^= 5	x = x ^ 5
*=	x *= 5	x = x * 5	>>=	x >>= 5	x = x >> 5
/=	x /= 5	x = x / 5	<<=	x <<= 5	x = x << 5
%=	x %= 5	x = x % 5			
//=	x //= 5	x = x // 5			
**=	x **= 5	x = x ** 5			

무단 수정 및 복제, 배포를 금지.

Python operators

- 항등(identity) 연산자
  - 두 개의 값(또는 변수)이 메모리의 동일한 부분에 있는지 확인하는데 사용
  - 두 변수가 동일하다고 이들이 동일하다는 의미는 아님

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]

# Output: False
print(x1 is not y1)

# Output: True
print(x2 is y2)

# Output: False
print(x3 is y3)
```

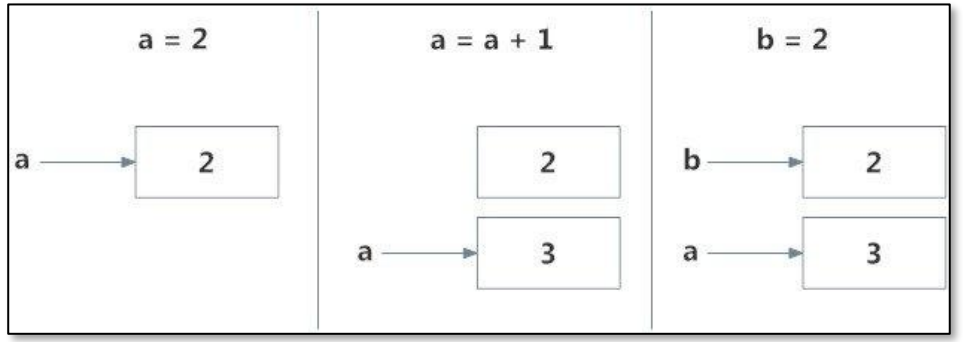
무단 수정 및 복제, 배포를 금지.

Name(identifier) in python

- Name(identifier) : 객체에 부여된 이름
  - 파이썬의 모든 것은 객체
  - 기본 개체에 접근하기 위한 방법

```
a = 2
print('id(2) =', id(2)) #output : 94618898111008
print('id(a) =', id(a)) #output : 94618898111008
```

```
a = 2
print('id(a) =', id(a)) #output : 94618898111008

a = a+1
print('id(a) =', id(a)) #output : 94618898111040

print('id(3) =', id(3)) #output : 94618898111040

b = 2
print('id(b) =', id(b)) #output : 94618898111008
print('id(2) =', id(2)) #output : 94618898111008
```



무단 수정 및 복제, 배포를 금지.

Name(identifier) in python

- Namespace
  - 변수의 적용 범위를 구분하여 사용
  - Function(local namespace) < module(global namespace) < built-in namespace

```
def outer_function():
    # b : local
    b = 20
    def inner_func():
        # c : nested local
        c = 30
        print(a)
    inner_func()

# a : global
a = 10
outer_function()
```

```
def outer_function():
    a = 20

    def inner_function():
        a = 30
        print('a =', a)

    inner_function()
    print('a =', a)


a = 10
outer_function()
print('a =', a)
```

```
def outer_function():
    global a
    a = 20

    def inner_function():
        global a
        a = 30
        print('a =', a)

    inner_function()
    print('a =', a)


a = 10
outer_function()
print('a =', a)
```

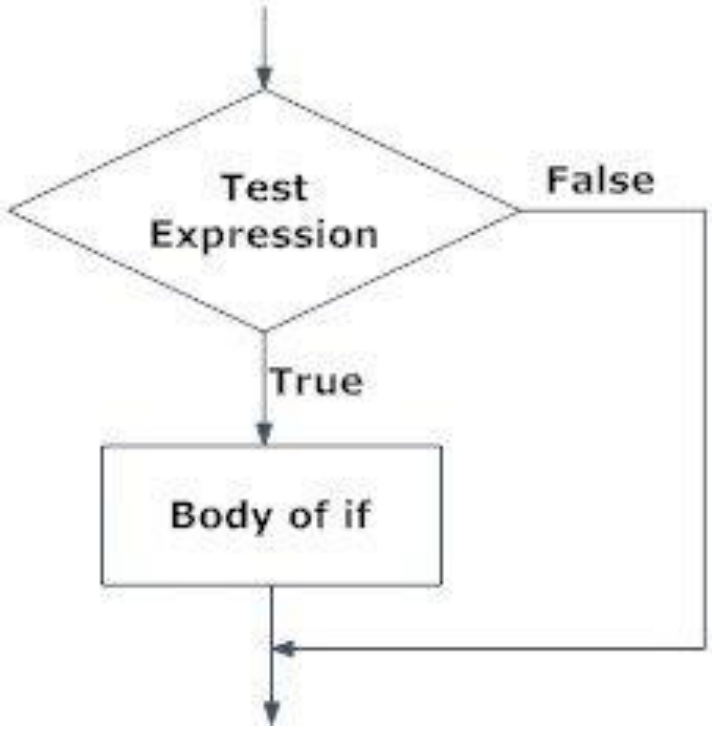
무단 수정 및 복제, 배포를 금지.

파이썬 흐름 제어 (flow control)

if

- 조건 만족하는 경우에만 실행
- 원형

```
if test expression:
    statement(s)
```



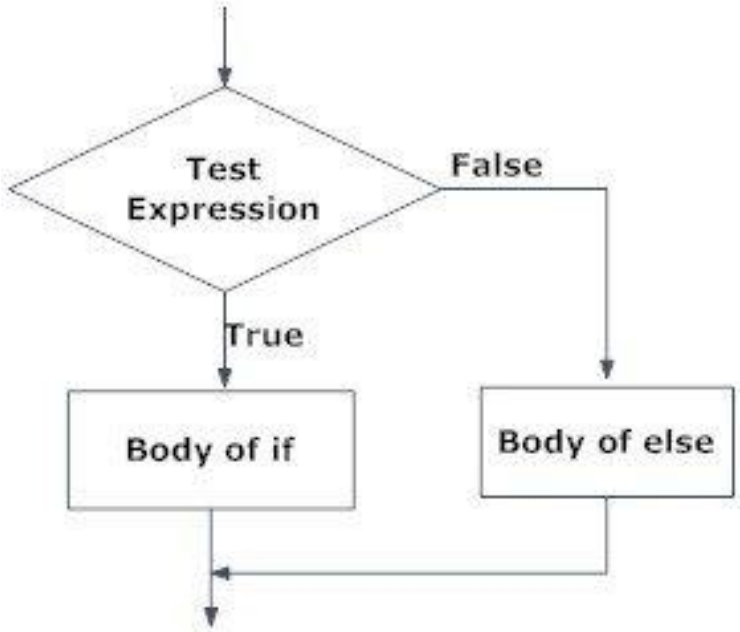

```
num = 3
if num > 0:
    print(num, "is a positive number.")
print("This is always printed.")

num = -1
if num > 0:
    print(num, "is a positive number.")
print("This is also always printed.")
```

무단 수정 및 복제, 배포를 금지.

if…else

- 조건 만족하는 경우와 아닌 경우를 구분하여 실행 내용이 달라짐
- 원형

```
if test expression:
    Body of if
else:
    Body of else
```



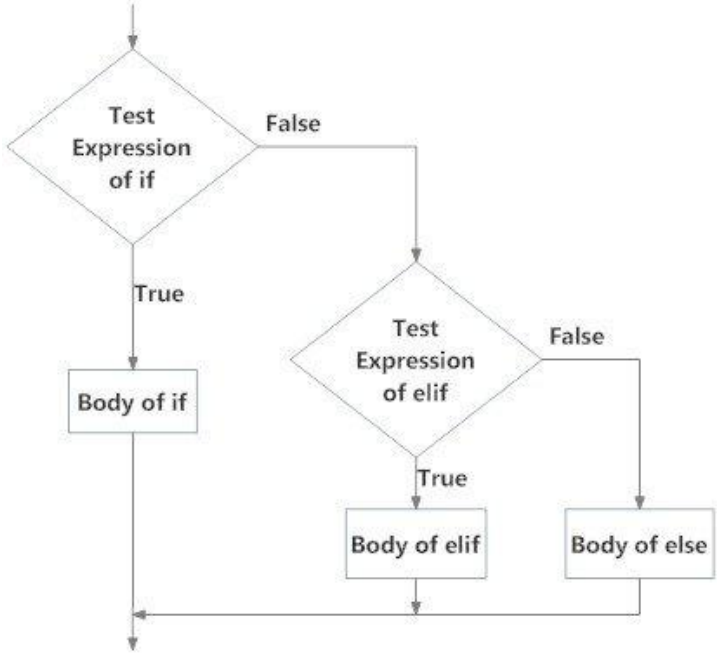

```
num = 3

if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

무단 수정 및 복제, 배포를 금지.

if…elif…else

- 여러 개의 조건 만족 여부를 판단하고 각 조건에 따라 실행 내용이 달라짐
- 원형

```
if test expression:
    Body of if
elif test expression:
    Body of elif
else:
    Body of else
```




```
num = 3.4

if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

무단 수정 및 복제, 배포를 금지.

Nested if…elif…else

- 조건 구문 내부에 다른 조건 구문을 둘 수 있음

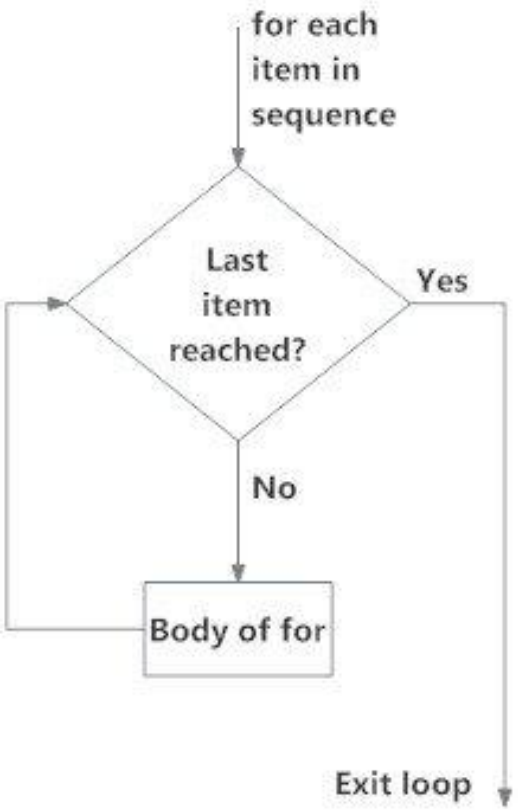
```
num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

무단 수정 및 복제, 배포를 금지.

for loop

- 반복 접근 가능한 객체/연산에 처리 과정에 사용
- 원형

```
for val in sequence:
    loop body
```




```
# List of numbers
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]

# variable to store the sum
sum = 0

# iterate over the list
for val in numbers:
    sum = sum+val

print("The sum is", sum)
```

무단 수정 및 복제, 배포를 금지.

for loop + range()

- range()
  - 숫자의 시퀀스를 생성하는 기능 제공 (생성되는 값을 모두 메모리에 저장하지는 않음)
  - 원형

```
range(start, stop, step_size)
```

  - in, len, __getitem__ 연산자 지원

```
# Output : range(0, 10)
print(range(10))

# Output : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(range(10)))

# Output : [2, 3, 4, 5, 6, 7]
print(list(range(2, 8)))

# Output : [2, 5, 8, 11, 14, 17]
print(list(range(2, 20, 3)))
```

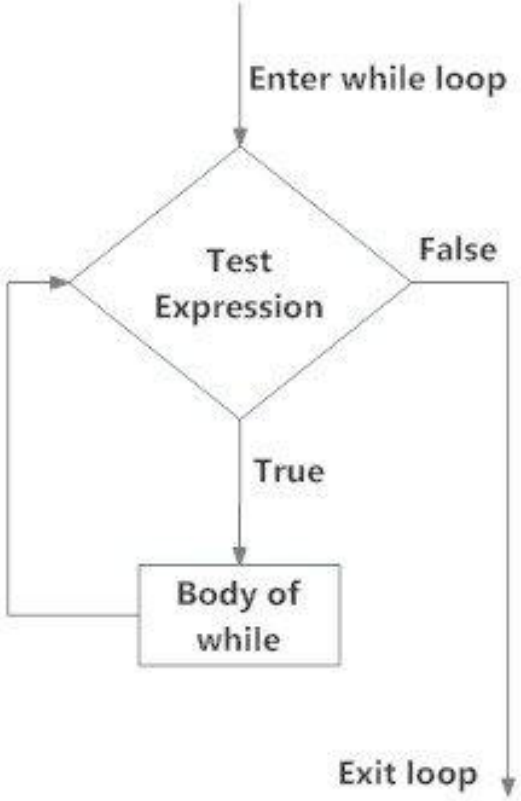
```
genre = ['pop', 'rock', 'jazz']

# iterate over the list using index
for i in range(len(genre)):
    print("I like", genre[i])
```

무단 수정 및 복제, 배포를 금지.

while loop, while loop + else

- while 루프는 조건을 만족한다면 계속 실행
- 원형

```
while test_expression:
    Body of while
```




```python
n = int(input("Enter n: "))

# initialize sum and counter
sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1     # update counter

# print the sum
print("The sum is", sum)
```

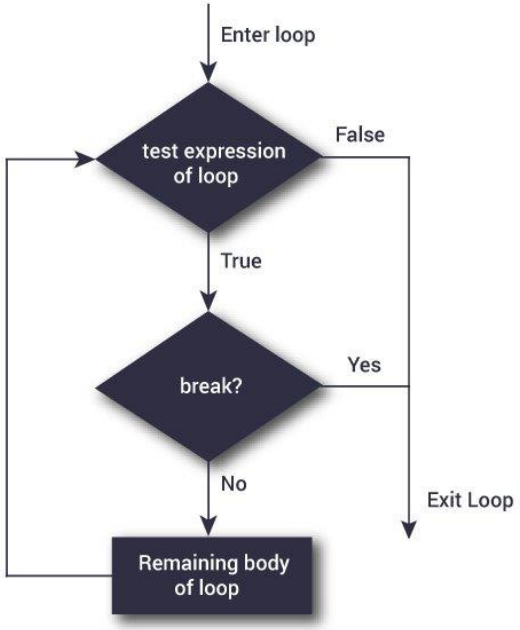
```python
counter = 0

while counter < 3:
    print("Inside loop")
    counter = counter + 1
else:
    print("Inside else")
```

무단 수정 및 복제, 배포를 금지.

break and continue

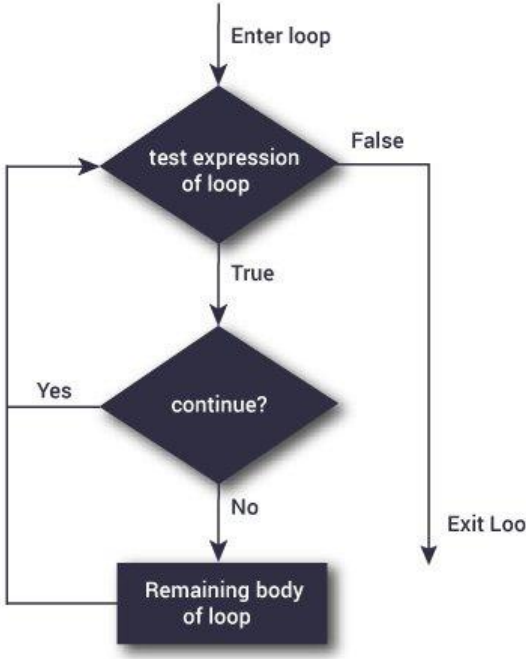
- break
  - 반복 루프를 종료 (루프 바로 다음의 명령문으로 실행 흐름 변경)
- continue
  - 현재 반복의 나머지 코드를 건너뛰어 루프 진행

```
for val in "string":
    if val == "i":
        # break
        # continue
    print(val)

print("The end")
```




```
for var in sequence:
    # codes inside for loop
    if  condition:
        break
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        break
    # codes inside while loop

# codes outside while loop
```




```
for var in sequence:
    # codes inside for loop
    if  condition:
        continue
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        continue
    # codes  inside while loop

# codes outside while loop
```

무단 수정 및 복제, 배포를 금지.

pass

- 빈 명령문(null statement)
- 인터프리터(interpreter)가 이 명령문을 무시하지는 않지만, 아무런 작업을 수행하지는 않음
  - 주석(comment)는 인터프리터가 그 자체를 무시한다는 것이 차이가 있음
- 추후에 구현되어야 하는 loop, function 등의 구현 부분에 사용
  - 선언만 되어 있고 구현 부분이 없으면 오류가 발생하는데, 이를 해결하기 위한 용도

```
sequence = {'p', 'a', 's', 's'}
for val in sequence:
    pass
```

```
def function(args):
    pass
```

```
class Example:
    pass
```

무단 수정 및 복제, 배포를 금지.

함수
(function)

function

- 특정 작업을 수행하기 위한 관련된 명령문(statement)의 그룹
- 프로그램을 더 작고, 모듈 청크(chunk)로 나눌 수 있음
- 코드의 중복성 낮추고, 재사용성을 높일 수 있음
- 선언 양식

```
def function_name(parameters):
    " " " docsting " " "
    statement(s)
```

- 사용하기 이전에 선언되어 있어야 함
- return 명령문 사용 가능함

```
return [expression_list]
```

- 변수의 적용 범위는 함수 내부와 외부가 별개임
- 사용자가 정의한 함수와 파이썬 내장 함수가 존재함

무단 수정 및 복제, 배포를 금지.

function

```
def greet(name):
    """This function greets to
    the person passed in as
    a parameter"""
    print("Hello, " + name + ". Good morning!")

greet('Paul')
```

```
def my_func():
    x = 10
    print("Value inside function:",x)

x = 20
my_func()
print("Value outside function:",x)
```

```
def absolute_value(num):
    """This function returns the absolute
    value of the entered number"""
    if num >= 0:
        return num
    else:
        return -num

num = absolute_value(2)
print(num)
print(absolute_value(-4))
```

무단 수정 및 복제, 배포를 금지.

Function arguments(인자)

- 함수를 사용하기 위해서는 선언된 인자에 관련 데이터를 전달해야 함

```
def greet(name, msg):
    """This function greets to
    the person with the provided message"""
    print("Hello", name + ', ' + msg)

greet("Monica", "Good morning!")
```

- 함수 사용시에, 전달하는 인자가 부족하거나 없다면 오류 발생함

```
TypeError: greet() missing 1 required positional argument: 'msg'

TypeError: greet() missing 2 required positional arguments: 'name' and 'msg'
```

무단 수정 및 복제, 배포를 금지.

Function arguments(인자)

- Default arguments
  - 선언시에, 인자의 기본값(default value)을 지정해줌
  - Default 이후에 오는 모든 인자는 기본값이 존재해야 함

```
def greet(name, msg="Good morning!"):
    """
    This function greets to
    the person with the
    provided message.

    If the message is not provided,
    it defaults to "Good
    morning!"
    """

    print("Hello", name + ', ' + msg)


greet("Kate")
greet("Bruce", "How do you do?")
```

```
def greet(msg = "Good morning!", name):
    print("Hello", name + ', ' + msg)


greet("Kate")
greet("Bruce", "How do you do?")

# Error
# SyntaxError: non-default argument follows
default argument
```

무단 수정 및 복제, 배포를 금지.

Function arguments(인자)

- Keyword arguments
  - 인자 전달시에, 위치 또는 키워드를 기반으로 적용
  - 키워드 인자 뒤에 위치를 사용하면 오류 발생

```
def greet(name, msg="Good morning!"):
    """
    This function greets to
    the person with the
    provided message.

    If the message is not provided,
    it defaults to "Good
    morning!"
    """

    print("Hello", name + ', ' + msg)
```

```
# 2 keyword arguments
greet(name = "Bruce",msg = "How do you do?")

# 2 keyword arguments (out of order)
greet(msg = "How do you do?",name = "Bruce")

# 1 positional, 1 keyword argument
greet("Bruce", msg = "How do you do?")

# Error
greet(name = "Bruce", "How do you do?")
```

무단 수정 및 복제, 배포를 금지.

Function arguments(인자)

- Arbitrary arguments
  - 인자에 대한 임의의 개수를 반영할 수 있음 (tuple 변환 일어남)
  - 별표(asterisk) *를 인자명 앞에 사용

```
def greet(*names):
    """This function greets all
    the person in the names tuple."""

    # names is a tuple with arguments
    for name in names:
        print("Hello", name)


greet("Monica", "Luke", "Steve", "John")
```

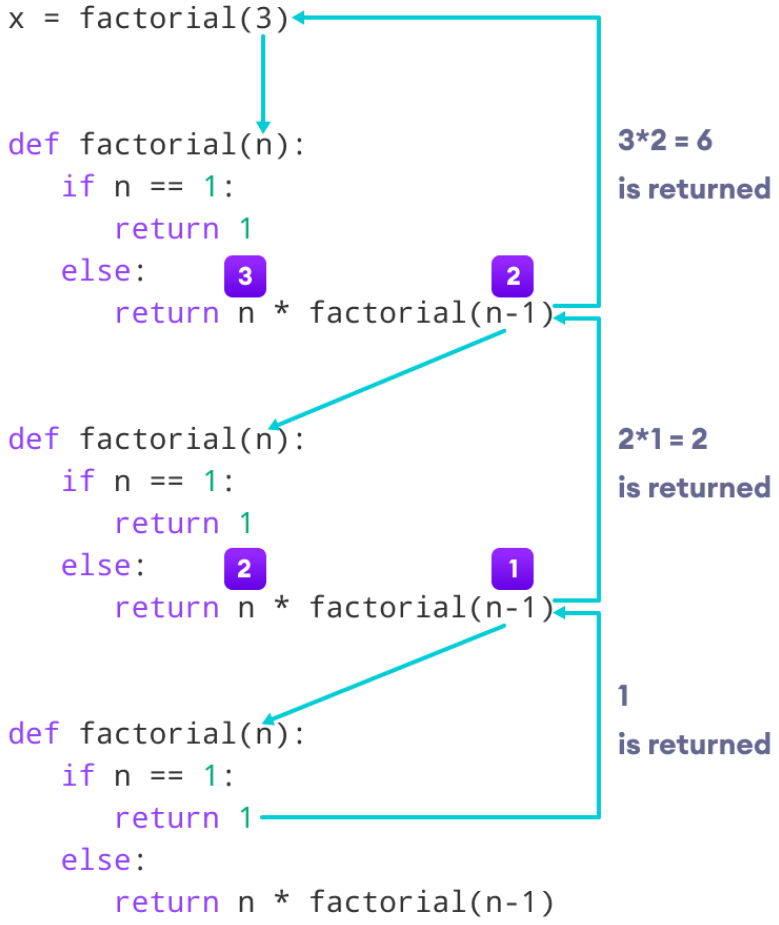
무단 수정 및 복제, 배포를 금지.

Recursion(재귀)

- 함수가 자기 자신을 호출하는 것
- 기본값으로, 최대 재귀 깊이(depth)는 1000임

```
def factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))

num = 3
print("The factorial of", num, "is", factorial(num))
```

factorial(3) # 1st call with 3
3 * factorial(2) # 2nd call with 2
3 * 2 * factorial(1) # 3rd call with 1
3 * 2 * 1 # return from 3rd call as number=1
3 * 2 # return from 2nd call
6 # return from 1st call



무단 수정 및 복제, 배포를 금지.

Anonymous/Lambda function

- 이름(name) 없이 정의된 함수(function)
- lambda 키워드를 사용하여 정의함
- 선언 방식 (하나의 expression 만 허용)

```
lamda argments: expression
```

```
double = lambda x: x * 2

print(double(5)
```

```
def double(x):
    return x * 2
```

- 단기간에 이름없는 함수가 필요할 때 사용함
  - filter(), map() 등의 내장 함수에 사용

```
my_list = [1, 5, 4, 6, 8, 11, 3, 12]
new_list = list(filter(lambda x: (x%2 == 0) , my_list))

print(new_list)
```

```
new_list = list(map(lambda x: x * 2 , my_list))

print(new_list)
```

무단 수정 및 복제, 배포를 금지.

Global, local and nonlocal

- Global variable
  - 함수의 외부 또는 global 범위에 선언된 변수
  - 함수의 내부 또는 외부에서 접근할 수 있음

```
x = "global"

def foo():
    print("x inside:", x)

foo()
print("x outside:", x)
```

```
x = "global"

def foo():
    x = x * 2
    print(x)

foo()
# Error
```

```
x = "global "

def foo():
    global x
    y = "local"
    x = x * 2
    print(x)
    print(y)

foo()
```

```
def foo():
    x = 20

    def bar():
        global x
        x = 25

    print("Before calling bar: ", x)
    print("Calling bar now")
    bar()
    print("After calling bar: ", x)

foo()
print("x in main: ", x)
```

무단 수정 및 복제, 배포를 금지.

Global, local and nonlocal

- Local variable
  - 함수의 내부 또는 local 범위에 선언된 변수

```
def foo():
    y = "local"
    print(y)

foo()
```

```
def foo():
    y = "local"

foo()
print(y)
# Error
```

- Nonlocal variable
  - Local 범위가 정의되지 않은 nested 함수에서 사용
  - Local 또는 global 범위에 있을 수 없음

```
def outer():
    x = "local"
    def inner():
        nonlocal x
        x = "nonlocal"
        print("inner:", x)
    inner()
    print("outer:", x)

outer()
```

무단 수정 및 복제, 배포를 금지.

Modules

- 명령문과 정의들이 포함되어 있는 파일을 참조
  - example.py
  - 함수를 파일에서 정의
- 관리 가능한 작은 파일로 분할 가능
- 코드의 재사용성 제공
- 파이썬 인터프리터가 모듈의 찾는 순서
  - 내장 모듈 → sys.path 상의 디렉토리
  - 현재 작업 폴더 → PYTHONPATH (환경 변수 상의 경로) → 설치 관련 기본 폴더

```
# Python Module : example.py

def add(a, b):
    """This program adds two
    numbers and return the result"""

    result = a + b
    return result
```

```
# Python Module : example.py

import example

example.add(4,5.5)
```

```
import math as m
print("The value of pi is", m.pi)
```

무단 수정 및 복제, 배포를 금지.

Modules

- renaming : **import** module_name **as** rename

```
import math as m
print("The value of pi is", m.pi)
```

- 모듈의 특정 이름(함수)만 가져오기 : **from** module_name **import** specific_name

```
from math import pi, e
print("The value of pi is", pi)
print("The value of e is", e)
```

- 모듈의 모든 이름(함수) 가져오기 : **from** module_name **import** *

```
from math import *
print("The value of pi is", pi)
```

무단 수정 및 복제, 배포를 금지.

Reloading a modules

- 기본적으로, 파이썬은 한번 불러온 모듈은 다시 불러오지 않음

```
# my_module.py

print("This code got executed")
```

```
import my_module
# Output : This code got executed
import my_module
import my_module
```

- imp 모듈의 reload() 함수 사용

```
import imp
import my_module
# Output : This code got executed
import my_module
imp.reload(my_module)
# Output : This code got executed
```

무단 수정 및 복제, 배포를 금지.

Find out names that are defined inside a module

- dir(module_name) : 특정 모듈에 정의되어 있는 이름을 나열

```
import math
dir(math)
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh',
'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh',
'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod',
'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf',
'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'pi',
'pow', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
```

- dir() : 현재 namespace 상에 정의되어 있는 이름을 나열

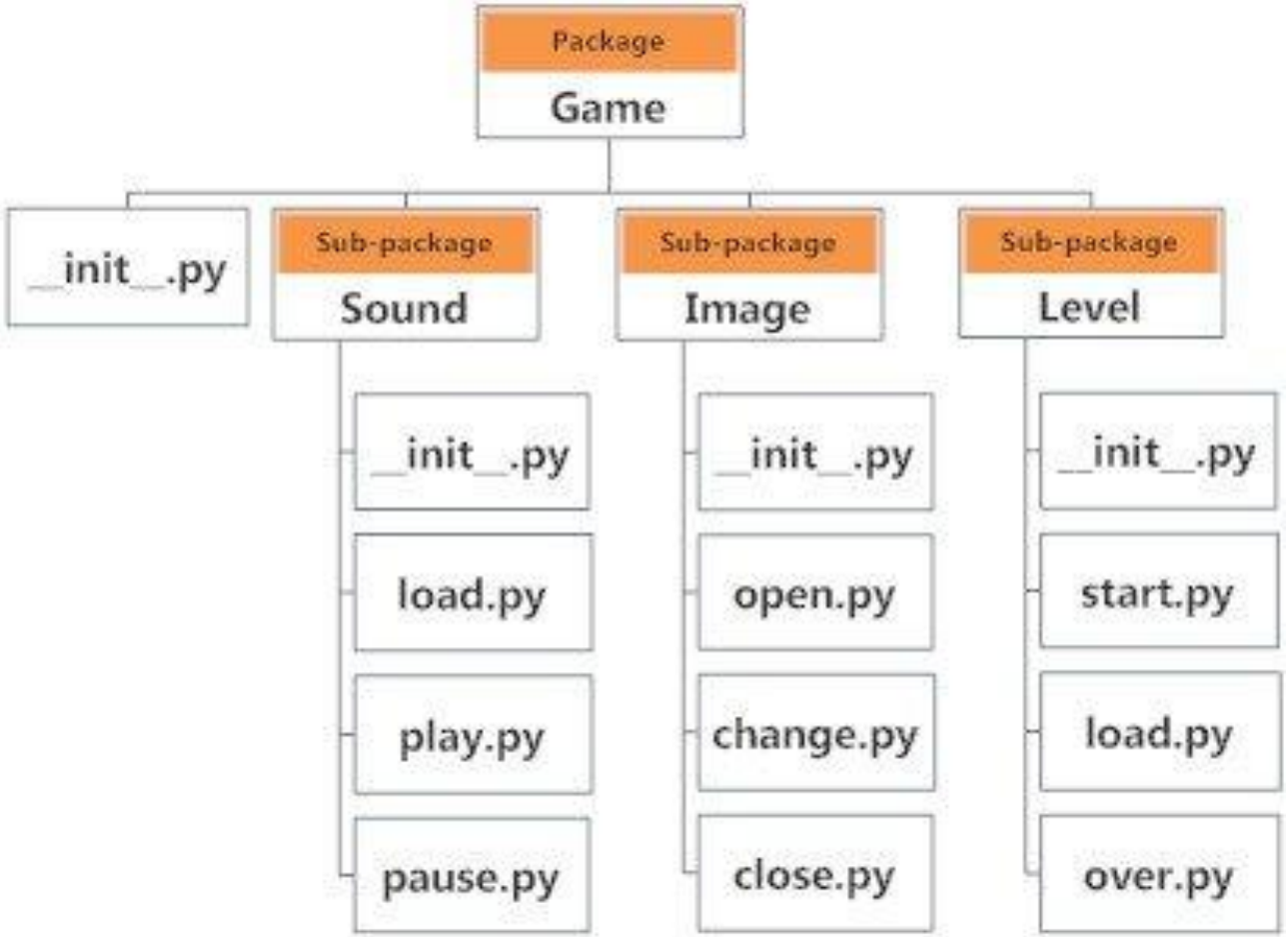
```
a = 1
b = "hello"
import math
dir()
```

```
['In', 'Out', '_', '__', '___', '__builtin__', '__builtins__', '__doc__',
'__loader__', '__name__', '__package__', '__spec__', '_dh', '_i', '_i1', '_ih', '_ii',
'_iii', '_oh', '_sh', 'a', 'b', 'exit', 'get_ipython', 'math', 'quit']
```

무단 수정 및 복제, 배포를 금지.

Python package

- 파이썬 프로젝트에 대한 폴더 단위 관리 지원



무단 수정 및 복제, 배포를 금지.

데이터형 (Datatypes)

Number data type

- int, float, complex
  - float : 소수점 이하 15자리까지 정확도 보장
- type() : 클래스의 데이터 형을 확인
- isinstance() : 주어진 값이 특정 클래스에 속하는지를 확인

```
a = 1
print(type(a))
print(isinstance(a, int))
```

- Type conversion
  - int(), float(), complex()
  - int와 float 간의 연산은 결과가 float로 강제 변환

무단 수정 및 복제, 배포를 금지.

Number data type

- Decimal(소수) 는 유한한 자리수까지 절삭되어 저장
  - 내장 함수인 decimal을 사용하면 더 정확한 값으로 계산
    - Float으로 사용하는 것보다는 속도가 느림
    - 금융 프로그램(정확한 십진수표현), 정밀도 필요, 유효 소수점 이하 구현 시에 사용

```
print((1.1 + 2.2) == 3.3)
print(1.1 + 2.2)
```

```
import decimal

# Output : 0.1
print(0.1)
# Output :
0.1000000000000000055511151231257827021181583404541015625
print(decimal.Decimal(0.1))
```

- fraction(분수)

```
import fractions
# Output : 3/2
print(fractions.Fraction(1.5))
# Output : 5
print(fractions.Fraction(5))
# Output : 1/3
print(fractions.Fraction(1,3))
```

무단 수정 및 복제, 배포를 금지.

Number data type

- Mathematics : math

Function	Description
ceil(x)	Returns the smallest integer greater than or equal to x.
copysign(x, y)	Returns x with the sign of y
fabs(x)	Returns the absolute value of x
factorial(x)	Returns the factorial of x
floor(x)	Returns the largest integer less than or equal to x
fmod(x, y)	Returns the remainder when x is divided by y
frexp(x)	Returns the mantissa and exponent of x as the pair (m, e)
fsum(iterable)	Returns an accurate floating point sum of values in the iterable
isfinite(x)	Returns True if x is neither an infinity nor a NaN (Not a Number)
isinf(x)	Returns True if x is a positive or negative infinity
isnan(x)	Returns True if x is a NaN
ldexp(x, i)	Returns x * (2**i)
modf(x)	Returns the fractional and integer parts of x
trunc(x)	Returns the truncated integer value of x
exp(x)	Returns e**x
expm1(x)	Returns e**x - 1
log(x[, b])	Returns the logarithm of x to the base b (defaults to e)
log1p(x)	Returns the natural logarithm of 1+x
log2(x)	Returns the base-2 logarithm of x
log10(x)	Returns the base-10 logarithm of x
pow(x, y)	Returns x raised to the power y
sqrt(x)	Returns the square root of x

Function	Description
acos(x)	Returns the arc cosine of x
asin(x)	Returns the arc sine of x
atan(x)	Returns the arc tangent of x
atan2(y, x)	Returns atan(y / x)
cos(x)	Returns the cosine of x
hypot(x, y)	Returns the Euclidean norm, sqrt(x*x + y*y)
sin(x)	Returns the sine of x
tan(x)	Returns the tangent of x
degrees(x)	Converts angle x from radians to degrees
radians(x)	Converts angle x from degrees to radians
acosh(x)	Returns the inverse hyperbolic cosine of x
asinh(x)	Returns the inverse hyperbolic sine of x
atanh(x)	Returns the inverse hyperbolic tangent of x
cosh(x)	Returns the hyperbolic cosine of x
sinh(x)	Returns the hyperbolic cosine of x
tanh(x)	Returns the hyperbolic tangent of x
erf(x)	Returns the error function at x
erfc(x)	Returns the complementary error function at x
gamma(x)	Returns the Gamma function at x
lgamma(x)	Returns the natural logarithm of the absolute value of the Gamma function at x
pi	Mathematical constant, the ratio of circumference of a circle to it's diameter (3.14159...)
e	mathematical constant e (2.71828...)

무단 수정 및 복제, 배포를 금지.

Number data type

- Mathematics : random

Function	Description
seed(a=None, version=2)	Initialize the random number generator
getstate()	Returns an object capturing the current internal state of the generator
setstate(state)	Restores the internal state of the generator
getrandbits(k)	Returns a Python integer with k random bits
randrange(start, stop[, step])	Returns a random integer from the range
randint(a, b)	Returns a random integer between a and b inclusive
choice(seq)	Return a random element from the non-empty sequence
shuffle(seq)	Shuffle the sequence
sample(population, k)	Return a k length list of unique elements chosen from the population sequence
random()	Return the next random floating point number in the range [0.0, 1.0)

Function	Description
uniform(a, b)	Return a random floating point number between a and b inclusive
triangular(low, high, mode)	Return a random floating point number between low and high, with the specified mode between those bounds
betavariate(alpha, beta)	Beta distribution
expovariate(lambd)	Exponential distribution
gammavariate(alpha, beta)	Gamma distribution
gauss(mu, sigma)	Gaussian distribution
lognormvariate(mu, sigma)	Log normal distribution
normalvariate(mu, sigma)	Normal distribution
vonmisesvariate(mu, kappa)	Vonmises distribution
paretovariate(alpha)	Pareto distribution
weibullvariate(alpha, beta)	Weibull distribution

무단 수정 및 복제, 배포를 금지.

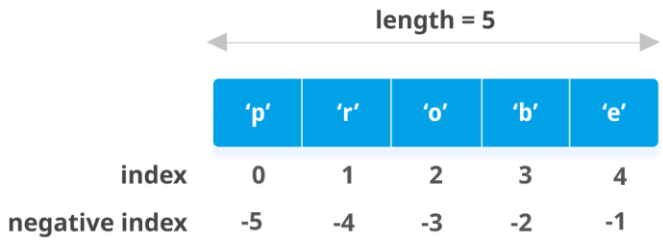
List

- 여러 요소를 한번에 작업할 수 있는 데이터 유형
- 각 요소에 순차적으로 접근할 수 있음
  - 인덱스는 0부터 시작하며, 정수만 가능함
  - 인덱스를 음수로 부여하면 끝에서부터 접근
  - slicing operator : 를 사용 지원

```
my_list = [1, 2, 3]
my_list = []
my_list = [1, "Hello", 3.4]
my_list = ["mouse", [8, 4, 6], ['a']]
```

```
# Nested List
n_list = [“probe", [2, 0, 1, 5]]
# Nested indexing
print(n_list[0][1])
print(n_list[1][3])
```

```
my_list = ['k','o','p','o','k','o','r','e','a']
# elements from index 2 to index 4
print(my_list[2:5])
# elements from index 5 to end
print(my_list[5:])
# elements beginning to end
print(my_list[:])
```



무단 수정 및 복제, 배포를 금지.

List

- 요소를 변경할 수 있음
- 요소를 추가할 수 있음
  - append() : 하나의 아이템 추가
  - extend() : 여러 개의 아이템 추가
  - insert() : 특정 위치에 아이템 추가
- List 간의 결합(combine)/연결(concatenation) 지원
  - +
- List를 반복 수행 지원
  - *

```
odd = [1, 3, 5]

odd[0] = 0
odd[1:3] = [2, 4]
print(odd)

odd[3:5] = [-5, -2]
print(odd)

odd.append(7)
print(odd)

odd.extend([9, 11, 13])
print(odd)

odd.insert(0,10)
print(odd)

print(odd + [9, 7, 5])

print(["re"] * 3)
```

무단 수정 및 복제, 배포를 금지.

List

- List 또는 요소 제거할 수 있음
  - del : 아이템 또는 list 자제 제거
  - remove : 특정 아이템 제거
  - pop : 특정 인덱스 제거(기본값은 -1)
  - clear : list 전체를 제거

```
# Deleting list items
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# delete one item, multiple items
del my_list[2]
print(my_list)

del my_list[1:5]
print(my_list)

# delete the entire list
del my_list

# Error: List not defined
print(my_list)
```

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')

# Output: ['r', 'o', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'o'
print(my_list.pop(1))

# Output: ['r', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'm'
print(my_list.pop())

# Output: ['r', 'b', 'l', 'e']
print(my_list)

my_list.clear()

# Output: []
print(my_list)
```

무단 수정 및 복제, 배포를 금지.

List

Function	Description
append()	adds an element to the end of the list
extend()	adds all elements of a list to another list
insert()	inserts an item at the defined index
remove()	removes an item from the list
pop()	returns and removes an element at the given index
clear()	removes all items from the list
index()	returns the index of the first matched item
count()	returns the count of the number of items passed as an argument
sort()	sort items in a list in ascending order
reverse()	reverse the order of items in the list
Copy()	returns a shallow copy of the list

```python
# Example on Python list methods

my_list = [3, 8, 1, 6, 8, 8, 4]

# Add 'a' to the end
my_list.append('a')

# Output: [3, 8, 1, 6, 8, 8, 4, 'a']
print(my_list)

# Index of first occurrence of 8
print(my_list.index(8))    # Output: 1

# Count of 8 in the list
print(my_list.count(8))   # Output: 3
```

무단 수정 및 복제, 배포를 금지.

List

- 제어 명령문에서 활용

```
for fruit in ['apple','banana','mango']:
    print("I like",fruit)
```

```
pow2 = [2 ** x for x in range(10)]
print(pow2)

pow2_1 = []
for x in range(10):
   pow2_1.append(2 ** x)
print(pow2_1)
```

무단 수정 및 복제, 배포를 금지.

List

- 제어 명령문에서 활용

```
odd = [x for x in range(20) if x % 2 == 1]
print(odd)

num_list = [y for y in range(100) if y % 2 == 0 if y % 5 == 0]
print(num_list)

[x+y for x in ['Python ','C '] for y in ['Lang','Prog']]
```

```
obj = ["Even" if i%2==0 else "Odd" for i in range(10)]
print(obj)
```

무단 수정 및 복제, 배포를 금지.

List

- 제어 명령문에서 활용

```
transposed = []
matrix = [[1, 2, 3, 4], [4, 5, 6, 8]]

for i in range(len(matrix[0])):
    transposed_row = []

    for row in matrix:
        transposed_row.append(row[i])
    transposed.append(transposed_row)

print(transposed)
```

```
matrix = [[1, 2], [3,4], [5,6], [7,8]]
transpose = \
[[row[i] for row in matrix] for i in range(2)]
print (transpose)
```

무단 수정 및 복제, 배포를 금지.

Tuple

- List와 대부분의 기능이 유사함
- 괄호 없이도 선언 가능하며, 쉼표 , 가 없으면 tuple로 인식하지 않음
- 선언 이후에는 내부 아이템을 변경할 수 없지만, 재할당은 가능함
  - 내부 아이템이 list인 경우에는 변경 가능
- 아이템만을 삭제할 수 없음

```
# Changing tuple values
my_tuple = (4, 2, 3, [6, 5])
# my_tuple = 4, 2, 3, [6, 5]

# TypeError: 'tuple' object does not support item assignment
# my_tuple[1] = 9

# However, item of mutable element can be changed
my_tuple[3][0] = 9    # Output: (4, 2, 3, [9, 5])
print(my_tuple)

# Tuples can be reassigned
my_tuple = ('k', 'o', 'p', 'd')

# Output: ('k', 'o', 'p', 'd')
print(my_tuple)
```

무단 수정 및 복제, 배포를 금지.

dictionary

- 다양한 방식으로 선언 및 접근 가능

```
# empty dictionary
my_dict = {}

# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}

# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}

# using dict()
my_dict = dict({1:'apple', 2:'ball'})

# from sequence having each item as a pair
my_dict = dict([(1,'apple'), (2,'ball')])
```

```
# get vs [] for retrieving elements
my_dict = {'name': 'Jack', 'age': 26}

# Output: Jack
print(my_dict['name'])

# Output: 26
print(my_dict.get('age'))

# Trying to access keys which doesn't exist
throws error
# Output None
print(my_dict.get('address'))

# KeyError
print(my_dict['address'])
```

무단 수정 및 복제, 배포를 금지.

dictionary

- Item 추가 및 변경, 삭제 가능

```
# Changing and adding Dictionary Elements
my_dict = {'name': 'Jack', 'age': 26}

# update value
my_dict['age'] = 27

#Output: {'age': 27, 'name': 'Jack'}
print(my_dict)

# add item
my_dict['add'] = 'Dt'

# Output: {'name': 'Jack', 'age': 27, 'add': 'Dt'}
print(my_dict)
```

```
# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))

# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

# remove an arbitrary item, return (key,value)
# Output: (5, 25)
print(squares.popitem())

# Output: {1: 1, 2: 4, 3: 9}
print(squares)

# remove all items
squares.clear()

# Output: {}
print(squares)

# delete the dictionary itself
del squares

# Throws Error
print(squares)
```

무단 수정 및 복제, 배포를 금지.

dictionary

- 다양한 표현 방법

```
squares = {x: x*x for x in range(6)}

print(squares)

'''
squares = {}
for x in range(6):
    squares[x] = x*x
print(squares)
'''
```

```
# Dictionary Comprehension with if conditional
odd_squares = {x: x*x for x in range(11) if x % 2 == 1}

print(odd_squares)
```

무단 수정 및 복제, 배포를 금지.

Dictionary methods

Method	Description
clear()	Removes all items from the dictionary.
copy()	Returns a shallow copy of the dictionary.
fromkeys(seq[, v])	Returns a new dictionary with keys from seq and value equal to v (defaults to None).
get(key[,d])	Returns the value of the key. If the key does not exist, returns d (defaults to None).
items()	Return a new object of the dictionary's items in (key, value) format.
keys()	Returns a new object of the dictionary's keys.
pop(key[,d])	Removes the item with the key and returns its value or d if key is not found. If d is not provid ed and the key is not found, it raises KeyError.
popitem()	Removes and returns an arbitrary item (**key, value**). Raises KeyError if the dictionary is empty.
setdefault(key[,d])	Returns the corresponding value if the key is in the dictionary. If not, inserts the key with a val ue of d and returns d (defaults to None).
update([other])	Updates the dictionary with the key/value pairs from other, overwriting existing keys.
values()	Returns a new object of the dictionary's values

무단 수정 및 복제, 배포를 금지.

Dictionary built-in functions

Method	Description
all()	Return True if all keys of the dictionary are True (or if the dictionary is empty).
any()	Return True if any key of the dictionary is true. If the dictionary is empty, return False.
len()	Return the length (the number of items) in the dictionary.
cmp()	Compares items of two dictionaries. (Not available in Python 3)
sorted()	Return a new sorted list of keys in the dictionary.

```
squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: False
print(all(squares))

# Output: True
print(any(squares))

# Output: 6
print(len(squares))

# Output: [0, 1, 3, 5, 7, 9]
print(sorted(squares))
```

무단 수정 및 복제, 배포를 금지.

File I/O

Open/close/write/read a file

- open(), close()
  - 작업 폴더를 기준으로 파일을 열거나 저장, 닫음
  - 파일명, 절대경로, 상대경로로 표시 가능

```
# opening a file
f = open("test.txt",'a') #f = open("c:\test.txt",'r')
f = open("test.txt", mode='r', encoding='utf-8')
f = open("test.txt", mode='r', encoding='cp1252')

#close a file
f.close()
```

```
try:
    f = open("test.txt", encoding = 'utf-8')
    # perform file operations
finally:
    f.close()
```

```
with open("test.txt", encoding = 'utf-8') as f:
    # perform file operations
```

Mode	Description
r	Opens a file for reading. (default)
w	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
x	Opens a file for exclusive creation. If the file already exists, the operation fails.
a	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
t	Opens in text mode. (default)
b	Opens in binary mode.
+	Opens a file for updating (reading and writing)

무단 수정 및 복제, 배포를 금지.

Open/close/write/read a file

- write()
  - Mode가 w, a, x, +조합인 경우에,

```
with open("test.txt",'w',encoding = 'utf-8') as f:
    f.write("my first file\n")
    f.write("This file\n\n")
    f.write("contains three lines\n")
```

- read()
  - Mode가 r, +조합인 경우에,

```
f = open("test.txt",'r',encoding = 'utf-8')
print(f.read(4))     # read the first 4 data
print(f.read(4))     # read the next 4 data
s1 = f.read()      # read in the rest till end of file
print(s1)
f.read()   # further reading returns empty sting
```

- tell(), seek()
  - 커서의 위치 반환 및 조정

```
print(f.tell())     # get the current file position
f.seek(0)    # bring file cursor to initial position
f.read()
```

무단 수정 및 복제, 배포를 금지.

Open/close/write/read a file

- 한줄씩(line-by-line) 읽어 오기
  - for loop 활용

```
f.seek(0)
for line in f:
    print(line, end = ' ')
```

  - readline(), readlines()

```
f.seek(0)
print(f.readline())
print(f.readline())
print(f.readline())
print(f.readline())

f.seek(0)
f.readlines()
```

무단 수정 및 복제, 배포를 금지.

Open/close/write/read a file

- File methods

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed.
detach()	Separates the underlying binary buffer from the TextIOBase and returns it.
fileno()	Returns an integer number (file descriptor) of the file.
flush()	Flushes the write buffer of the file stream.
isatty()	Returns True if the file stream is interactive.
read(n)	Reads at most n characters from the file. Reads till end of file if it is negative or None.
readable()	Returns True if the file stream can be read from.
readline(n=-1)	Reads and returns one line from the file. Reads in at most n bytes if specified.
readlines(n=-1)	Reads and returns a list of lines from the file. Reads in at most n bytes/characters if specified.
seek(offset,from=SEEK_SET)	Changes the file position to offset bytes, in reference to from (start, current, end).
seekable()	Returns True if the file stream supports random access.
tell()	Returns an integer that represents the current position of the file's object.
truncate(size=None)	Resizes the file stream to size bytes. If size is not specified, resizes to current location.
writable()	Returns True if the file stream can be written to.
write(s)	Writes the string s to the file and returns the number of characters written.
writelines(lines)	Writes a list of lines to the file.

무단 수정 및 복제, 배포를 금지.

Directory and files management

- os module
  - getcwd(), getcwdb() : 현재 작업 폴더 확인

```
import os

print(os.getcwd())   #/content
print(os.getcwdb()) #b'/content'
os.getcwd()
```

  - chdir() : 작업 폴더 변경

```
os.chdir('../home')

print(os.getcwd()) #/home
```

  - listdir() : 현재 폴더 내부의 파일 및 하위 폴더 확인
  - mkdir(), makedirs() : 폴더 생성
  - rename() : 파일, 폴더의 이름 변경
  - remove() : 파일 삭제
  - rmdir(), shutill.rmtree() : 폴더 삭제

```
import os

# ['.config', 'sample_data']
print(os.listdir())

path = 'test'
if not os.path.isdir(path):
        os.mkdir(path)
# ['.config', 'test', 'sample_data']
print(os.listdir())

os.rename('test', 'new_test')
# ['.config', 'new_test', 'sample_data']
print(os.listdir())

os.rmdir('new_test')
# ['.config', 'sample_data']
print(os.listdir())

import shutil
shutil.rmtree('test')
```

무단 수정 및 복제, 배포를 금지.

Error and exceptions

- Syntax error(parsing error) : 문법적인 오류

```
if a < 3

'''
  File "<ipython-input-11-3e28e520013d>", line 1
    if a < 3
            ^
SyntaxError: invalid syntax
'''
```

- Logical error(exception) : 실행 중에 발생하는 오류

```
a = 1 / 0

'''
ZeroDivisionError                         Traceb
ack (most recent call last)
<ipython-input-12-6a8f83430dbe> in <module>()
----> 1 a = 1 / 0

ZeroDivisionError: division by zero
'''
```

무단 수정 및 복제, 배포를 금지.

Error and exceptions

- Syntax error(parsing error) : 문법적인 오류

```
if a < 3

'''
  File "<ipython-input-11-3e28e520013d>", line 1
    if a < 3
            ^
SyntaxError: invalid syntax
'''
```

- Logical error(exception) : 실행 중에 발생하는 오류

```
a = 1 / 0

'''
ZeroDivisionError                         Traceb
ack (most recent call last)
<ipython-input-12-6a8f83430dbe> in <module>()
----> 1 a = 1 / 0

ZeroDivisionError: division by zero
'''
```

무단 수정 및 복제, 배포를 금지.

Error and exceptions

- Built-in exceptions (`print(dir(locals()['__builtins__']))`)

Exception	Cause of error
AssertionError	Raised when an assert statement fails.
AttributeError	Raised when attribute assignment or reference fails.
EOFError	Raised when the input() function hits end-of-file condition.
FloatingPointError	Raised when a floating point operation fails.
GeneratorExit	Raise when a generator's close() method is called.
ImportError	Raised when the imported module is not found.
IndexError	Raised when the index of a sequence is out of range.
KeyError	Raised when a key is not found in a dictionary.
KeyboardInterrupt	Raised when the user hits the interrupt key (Ctrl+C or Delete).
MemoryError	Raised when an operation runs out of memory.
NameError	Raised when a variable is not found in local or global scope.
NotImplementedError	Raised by abstract methods.
OSError	Raised when system operation causes system related error.
OverflowError	Raised when the result of an arithmetic operation is too large to be represented.

무단 수정 및 복제, 배포를 금지.

Error and exceptions

Exception	Cause of error
ReferenceError	Raised when a weak reference proxy is used to access a garbage collected referent.
RuntimeError	Raised when an error does not fall under any other category.
StopIteration	Raised by next() function to indicate that there is no further item to be returned by iterator.
SyntaxError	Raised by parser when syntax error is encountered.
IndentationError	Raised when there is incorrect indentation.
TabError	Raised when indentation consists of inconsistent tabs and spaces.
SystemError	Raised when interpreter detects internal error.
SystemExit	Raised by sys.exit() function.
TypeError	Raised when a function or operation is applied to an object of incorrect type.
UnboundLocalError	Raised when a reference is made to a local variable in a function or method, but no value has been bound to that variable.
UnicodeError	Raised when a Unicode-related encoding or decoding error occurs.
UnicodeEncodeError	Raised when a Unicode-related error occurs during encoding.
UnicodeDecodeError	Raised when a Unicode-related error occurs during decoding.
UnicodeTranslateError	Raised when a Unicode-related error occurs during translating.
ValueError	Raised when a function gets an argument of correct type but improper value.
ZeroDivisionError	Raised when the second operand of division or modulo operation is zero.

무단 수정 및 복제, 배포를 금지.

Exception handling

- try, except

```
# import module sys to get the type of exception
import sys

randomList = ['a', 0, 2]

for entry in randomList:
    try:
        print("The entry is", entry)
        r = 1/int(entry)
        break
    except:
        print("Oops!", sys.exc_info()[0], "occurred.")
        print("Next entry.")
        print()
print("The reciprocal of", entry, "is", r)
```

```
except Exception as e:
    print("Oops!", e.__class__, "occurred.")
    print("Next entry.")
    print()
```

무단 수정 및 복제, 배포를 금지.

Exception handling

- try, except, raise, else

```
try:
    # do something
    pass

except ValueError:
    # handle ValueError exception
    pass

except (TypeError, ZeroDivisionError):
    # handle multiple exceptions
    # TypeError and ZeroDivisionError
    pass

except:
    # handle all other exceptions
    pass
```

```
try:
    a = int(input("Enter a positive integer: "))
    if a <= 0:
        raise ValueError("That is not a positive number!")
except ValueError as ve:
    print(ve)
```

```
# program to print the reciprocal of even numbers

try:
    num = int(input("Enter a number: "))
    assert num % 2 == 0, "error message of assert"
except:
    print("Not an even number!")
else:
    reciprocal = 1/num
    print(reciprocal)
```

무단 수정 및 복제, 배포를 금지.

Customizing exception classes

- 사용자가 필요에 의해 exception을 발생시킴
- 하나의 파일(exceptions.py, errors.py)에서 관리 추천

```
class CustomError(Exception):
    print('custom exception’)

#raise CustomError("An error occurred")
#raise CustomError
try:
    raise CustomError
except:
    pass
```

```
# define Python user-defined exceptions
class Error(Exception):
    """Base class for other exceptions"""
    pass

class ValueTooSmallError(Error):
    """Raised when the input value is too small"""
    pass

class ValueTooLargeError(Error):
    """Raised when the input value is too large"""
    pass

# you need to guess this number
number = 10

# user guesses a number until he/she gets it right
while True:
    try:
        i_num = int(input("Enter a number: "))
        if i_num < number:
            raise ValueTooSmallError
        elif i_num > number:
            raise ValueTooLargeError
        break
    except ValueTooSmallError:
        print("This value is too small, try again!")
        print()
    except ValueTooLargeError:
        print("This value is too large, try again!")
        print()

print("Congratulations! You guessed it correctly.")
```

무단 수정 및 복제, 배포를 금지.

Custom exceptions

- 사용자가 필요에 의해 exception을 발생시킴

```
class SalaryNotInRangeError(Exception):
    """Exception raised for errors in the input salary.

    Attributes:
        salary -- input salary which caused the error
        message -- explanation of the error
    """

    def __init__(self, salary, message="Salary is
 not in (5000, 15000) range"):
        self.salary = salary
        self.message = message
        super().__init__(self.message)


salary = int(input("Enter salary amount: "))
if not 5000 < salary < 15000:
    raise SalaryNotInRangeError(salary)
```

```
class SalaryNotInRangeError(Exception):
    """Exception raised for errors in the input salary.

    Attributes:
        salary -- input salary which caused the error
        message -- explanation of the error
    """

    def __init__(self,
                 salary,
                 message="Salary is not in (5000, 15
000) range"):
        self.salary = salary
        self.message = message
        super().__init__(self.message)

    def __str__(self):
        return f'{self.salary} -> {self.message}'


salary = int(input("Enter salary amount: "))
if not 5000 < salary < 15000:
    raise SalaryNotInRangeError(salary)
```

무단 수정 및 복제, 배포를 금지.

Object & class

Object-oriented programming(OOP) (객체 지향 프로그래밍)

- 파이썬은 다중 패러다임 프로그래밍 언어
  - 다양한 프로그래밍 접근 방식 지원
- 재사용 가능한 코드 작성에 초점 (DRY, Don't Repeat Yourself)
- Class는 속성(attribute)과 동작(behavior)에 대한 정의
  - class 키워드를 이용하여 정의
- Object는 class의 인스턴스(instance)

```
class Parrot:
    pass

obj = Parrot()
```

```
class Human:

    # class attribute
    species = "human"

    # instance attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age

# instantiate the Parrot class
kim = Human("kim", 10)
lee = Human("lee", 15)

# access the class attributes
print("kim is a {}".format(kim.__class__.species))
print("lee is also a {}".format(lee.__class__.species))

# access the instance attributes
print("{} is {} years old".format( kim.name, kim.age))
print("{} is {} years old".format( lee.name, lee.age))
```

무단 수정 및 복제, 배포를 금지.

Object-oriented programming(OOP) (객체 지향 프로그래밍)

- Method
  - 함수에 정의되어 있는 함수
  - 객체(object)의 동작(behavior)을 정의

```
class Human:

    # instance attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # instance method
    def sing(self, song):
        return "{} sings {}".format(self.name, song)

    def dance(self):
        return "{} is now dancing".format(self.name)

# instantiate the Parrot class
kim = Human("kim", 10)

# call our instance methods
print(kim.sing("'Happy'"))
print(kim.dance())
```

무단 수정 및 복제, 배포를 금지.

Object-oriented programming(OOP) (객체 지향 프로그래밍)

- 상속(Inheritance)
  - 기본(상위) 클래스의 내용을 수정하지 않고, 그대로 사용하기 위해 새로운 파생(자식) 클래스를 생성하는 방식

```
# parent class
class Bird:
    def __init__(self):
        print("Bird is ready")
    def whoisThis(self):
        print("Bird")
    def swim(self):
        print("Swim faster")

# child class
class Penguin(Bird):
    def __init__(self):
        # call super() function
        super().__init__()
        print("Penguin is ready")
    def whoisThis(self):
        print("Penguin")
    def run(self):
        print("Run faster")

peggy = Penguin()
peggy.whoisThis()
peggy.swim()
peggy.run()
```

무단 수정 및 복제, 배포를 금지.

Object-oriented programming(OOP) (객체 지향 프로그래밍)

- 캡슐화(Encapsulation)
  - Method, 변수에 대한 접근 제한
  - 직접적인 데이터 수정 방지

```
class Computer:

    def __init__(self):
        self.__maxprice = 900

    def sell(self):
        print("Selling Price: {}".format(self.__maxprice))

    def setMaxPrice(self, price):
        self.__maxprice = price

c = Computer()
c.sell()

# change the price
c.__maxprice = 1000
c.sell()

# using setter function
c.setMaxPrice(1000)
c.sell()
```

무단 수정 및 복제, 배포를 금지.

Object-oriented programming(OOP) (객체 지향 프로그래밍)

- Polymorphism (다형성)
  - 다양한 데이터 유형에 대해 공통 인터페이스를 사용

```
class Parrot:
    def fly(self):
        print("Parrot can fly")

    def swim(self):
        print("Parrot can't swim")

class Penguin:
    def fly(self):
        print("Penguin can't fly")

    def swim(self):
        print("Penguin can swim")

# common interface
def flying_test(bird):
    bird.fly()

#instantiate objects
blu = Parrot()
peggy = Penguin()

# passing the object
flying_test(blu)
flying_test(peggy)
```

무단 수정 및 복제, 배포를 금지.

Class

- 생성 방법
  - class 키워드 사용하여 정의
  - 정의하면, 동일한 이름으로 새 클래스 객체가 생성됨

```
class MyNewClass:
    '''This is a docstring. I have created a
new class'''
    pass
```

```
class Person:
    "This is a person class"
    age = 10
    def greet(self):
        print('Hello')


# Output: 10
print(Person.age)

# Output: <function Person.greet>
print(Person.greet)

# Output: "This is a person class"
print(Person.__doc__)
```

```
class ComplexNumber:
    def __init__(self, r=0, i=0):
        self.real = r
        self.imag = i

    def get_data(self):
        print(f'{self.real}+{self.imag}j')


# Create a new ComplexNumber object
num1 = ComplexNumber(2, 3)

# Call get_data() method
# Output: 2+3j
num1.get_data()

# Create another ComplexNumber object
# and create a new attribute 'attr'
num2 = ComplexNumber(5)
num2.attr = 10

# Output: (5, 0, 10)
print((num2.real, num2.imag, num2.attr))

print(num1.attr)
```

무단 수정 및 복제, 배포를 금지.

Class

- 클래스, 속성(variables), 동작(methods) 제거
  - del 키워드 사용

```
num1 = ComplexNumber(2,3)
del num1.imag
num1.get_data()
```

```
num1 = ComplexNumber(2,3)
del ComplexNumber.get_data
num1.get_data()
```

```
num1 = ComplexNumber(2,3)
del num1
```

  - 실제로는 메모리상에 계속 존재하며, garbage collection에 의해 나중에 자동 소멸

무단 수정 및 복제, 배포를 금지.

Class

- 상속
  - isinstance(), issubclass()를 활용하여 상속 여부를 확인하는데 사용

```
class Polygon:
    def __init__(self, no_of_sides):
        self.n = no_of_sides
        self.sides = [0 for i in range(no_of_sides)]
    def inputSides(self):
        self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in range(self.n)]
    def dispSides(self):
        for i in range(self.n):
            print("Side",i+1,"is",self.sides[i])
```

```
class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)
        #super().__init__(3)

    def findArea(self):
        a, b, c = self.sides
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('The area of the triangle is %0.2f' %area)
```

```
t = Triangle()
t.inputSides()
t.dispSides()
t.findArea()
```

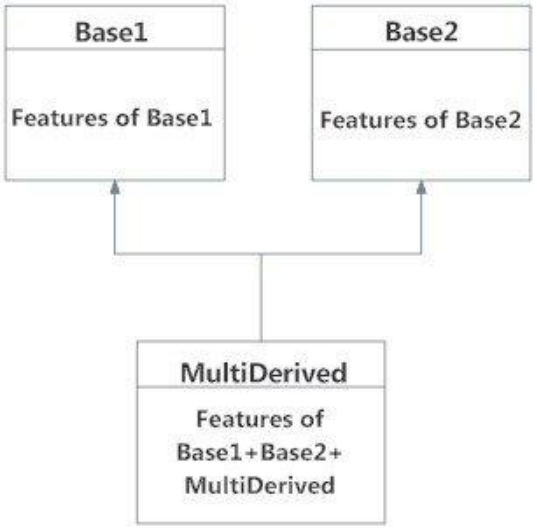
무단 수정 및 복제, 배포를 금지.

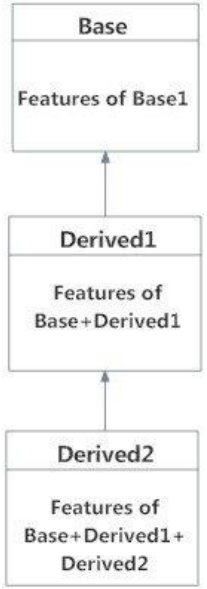
Class

- 다중 상속(multiple inheritance)
  - __mro__, mro()를 이용하여 MRO(method resolution order) 클래스 선형화의 순서를 찾을 수 있음

```
class Base1:
    pass

class Base2:
    pass

class MultiDerived(Base1, Base2):
    pass
```

```
class Base:
    pass

class Derived1(Base):
    pass

class Derived2(Derived1):
    pass
```





무단 수정 및 복제, 배포를 금지.

Class

- Overloading

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(2, 3)
print(p1+p2)
```

- 특수 함수(special functions) 이용
  - Double underscore(__)로 이용하여 시작하는 클래스 함수
    - 예) __init__()

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
    def __str__(self):
        return "({0}, {1})".format(self.x, self.y)

p1 = Point(2, 3)
print(p1)
```

```
print(str(p1))
print(format(p1))
```

무단 수정 및 복제, 배포를 금지.

Class

- 특수 함수(special functions) 이용

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)


p1 = Point(1, 2)
p2 = Point(2, 3)

print(p1+p2)
print(Point.__add__(p1,p2))
```

무단 수정 및 복제, 배포를 금지.

Class

Operator	Expression	Internally
Addition	p1 + p2	p1.__add__(p2)
Subtraction	p1 - p2	p1.__sub__(p2)
Multiplication	p1 * p2	p1.__mul__(p2)
Power	p1 ** p2	p1.__pow__(p2)
Division	p1 / p2	p1.__truediv__(p2)
Floor Division	p1 // p2	p1.__floordiv__(p2)
Remainder (modulo)	p1 % p2	p1.__mod__(p2)
Bitwise Left Shift	p1 << p2	p1.__lshift__(p2)
Bitwise Right Shift	p1 >> p2	p1.__rshift__(p2)
Bitwise AND	p1 & p2	p1.__and__(p2)
Bitwise OR	p1 | p2	p1.__or__(p2)
Bitwise XOR	p1 ^ p2	p1.__xor__(p2)
Bitwise NOT	~p1	p1.__invert__()

Operator	Expression	Internally
Less than	p1 < p2	p1.__lt__(p2)
Less than or equal to	p1 <= p2	p1.__le__(p2)
Equal to	p1 == p2	p1.__eq__(p2)
Not equal to	p1 != p2	p1.__ne__(p2)
Greater than	p1 > p2	p1.__gt__(p2)
Greater than or equal to	p1 >= p2	p1.__ge__(p2)

무단 수정 및 복제, 배포를 금지.